深圳大学 SHENZHEN UNIVERSITY

课程 COURSE

# Big Data Analytics
# 大数据分析 [2026 国际商务]

## Lecture 5 — Introduction to Machine Learning 第五讲·机器学习入门
## 从学习范式到评估协议、算法族与可信交付

任课教师：戴谢尔 (Xieer Dai, David)

深圳大学 Shenzhen University

上课时间：第 3–14 周 星期四 11-12 节

上课地点：四方楼南 209

# 学习目标 Learning Objectives

完成本讲后，你应能够：

1. 区分监督、无监督、自监督与强化学习四类范式，并说明各自需要什么数据；
2. 用偏差-方差与三类误差解释过拟合与欠拟合，并读懂学习曲线；
3. 正确执行训练 / 验证 / 测试划分与交叉验证，识别六类数据泄漏；
4. 选择并评估基线模型：回归与分类指标、阈值与业务代价、概率校准；
5. 说清神经网络与强化学习的基本机制（前向传播、激活函数、反向传播、奖励）；
6. 交付一份可信的模型：可解释性、模型卡、数据泄漏清单，并写出项目选题说明（问题 / 数据 / 方法）。

来源：对应课程大纲第 5 讲目标：区分学习范式；理解偏差-方差与评估协议；能建立并评估基线模型。

# 本讲路线图 Roadmap（126 帧）




**主线**：概念 → 工具 → 流程 → 评估 → 算法族 → 可信交付

**代码页 11 帧·样本页 22 帧**：全部示例用 Python，每一行代码都带英文注释，随机种子固定 20260506

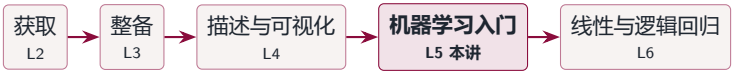
# 与上一讲的衔接 From Description to Prediction

## L4 结束时你手上有什么

- 干净的数据表 + 数据字典；
- 描述统计表与图表清单；
- 但还没有回答：能不能预测？预测得准不准？凭什么相信它？

## L5 要回答的问题

- 这个问题属于哪类学习范式？
- 怎么划数据、怎么衡量好坏？
- 该选哪个算法族，代价是什么？
- 结论怎么解释、记录与交付？




本讲是总览课：把四大范式、评估协议与主要算法族串成一张地图；L6–L9 再逐个深入。

# 本讲输出物 Deliverables

**1) 基线模型与评估协议**

划分方式、交叉验证、指标与阈值选择，全部落盘成表（样本页 28–30、44 页）

**2) 数据泄漏清单**

目标泄漏、选择泄漏、时间泄漏、重复行、分组泄漏逐条自检（93 页）

**3) 模型卡与运行记录**

用途、数据、指标、限制、伦理；一键重跑的 JSON 记录（92 页代码）

**与第 6 周任务对应：项目选题说明（一页）**

问题（预测还是因果）/ 数据（来源与许可）/ 方法（基线 + 评估协议 + 指标）/ 预期产出。帧 101 给出填写模板，本讲的评估协议直接照抄。

来源：评分与提交要求见课程大纲第六节；本讲提供模板与检查清单。

# 全讲导览 Map of the Lecture

| 部分 | 帧 | 关键问题 |
|---|---|---|
| **概念与版图** | 7–18 | 什么是机器学习？四类范式各要什么数据？ |
| **流程、划分与评估协议** | 19–30 | 怎么划数据？预测与因果怎么分工？ |
| **泛化、偏差-方差与评估** | 31–46 | 为什么样本外会变差？该报哪个指标？ |
| **监督学习算法族** | 47–61 | 回归、KNN、SVM、树与集成怎么选？ |
| **无监督学习算法族** | 62–73 | 没有标签时怎么降维、怎么聚类、怎么评估？ |
| **神经网络与强化学习** | 74–87 | 前向、反向、激活函数与奖励到底在算什么？ |
| **可信、可解释与工程化** | 88–99 | 怎么解释、怎么记录、怎么避免泄漏与偏差？ |
| **项目与结语** | 100–104 | 第 6 周的选题说明怎么写？ |

读法：每部分都是" 概念 → 样本与图示（样本页）→ 可运行程序（代码页）→ 检查清单" 的循环。

# 从三个问题切入 Three Questions

## 预测一个数

" 下个月这家门店的销售额是多少？"
任务：回归；指标：RMSE / MAE

## 判断一个类

" 这笔交易是不是欺诈？"
任务：分类；指标：AUC / 召回率

## 找出一些组

" 哪些客户行为相似？"
任务：聚类；指标：轮廓系数 / 业务解释

## 三个问题对应三种" 能不能被检验"

- 回归与分类有标签，可以用样本外误差检验；
- 聚类没有标签，" 好" 取决于用途与解释——评估最容易被忽略；
- 三类任务共同的纪律：先划数据，再谈模型。

来源：三类任务的划分据统计学习通用框架；与课程项目选题（问题/数据/方法）直接对应。

# 机器学习的定义 Mitchell (1997)

## 定义

"A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E."

— Tom Mitchell, *Machine Learning*, 1997

| 要素 | 含义 | 信用卡欺诈场景 |
|---|---|---|
| **T 任务** | 要完成的事情 | 判断一笔交易是否为欺诈 |
| **E 经验** | 用于学习的数据 | 历史交易与人工标注结果 |
| **P 性能度量** | 判断好坏的标准 | ROC-AUC、召回率、误报成本 |

定义的威力在于把" 学习" 拆成可检查的三件事：任务说清了没有？经验够不够、有没有偏？度量选对了吗？——本讲后面每一部分都在回答其中一个。

来源：定义与出处：Mitchell, T. (1997). *Machine Learning*, McGraw-Hill。

# AI、机器学习、深度学习与生成式 AI The Landscape







Not magic: a function fitted on data, with assumptions you can inspect.

## 边界在哪

- ML 是" 从数据中学一个函数"：给定输入预测输出；
- 深度学习是 ML 的一支：用多层网络表示复杂函数；
- 生成式 AI 学习的是数据分布，输出可以不是标签而是内容（衔接 L11）。

来源：版图据统计学习与基础模型研究的通行划分；与 L1 的 AI 版图、L11 的 NLP 内容衔接。

# 金融业的应用 Applications in Finance

## 投资管理价值链

技术已经渗透到每一步：更好地理解客户 → 发现新的 alpha 来源 → 更高效地执行交易。机器学习技术最早于 1990 年代进入金融，随着数据爆炸与廉价算力而繁荣。

## 从业者的角色

投资从业者不必精通数学细节，但作为领域专家，重要的是能够：选择合适的模型输入、解读模型输出、把输出转化为投资动作。

## 七类典型应用

1. 流程自动化
2. 文档分析
3. 组合管理
4. 算法交易
5. 数字助理
6. 风险管理
7. 欺诈与反洗钱

来源：应用清单与价值链表述据金融机器学习教材中的行业综述；本讲按课程语境整理。

# 案例：用年报文本预测收益 Filings as Data

## 一个真实的研究范式

研究者用文本分析发现：年报（10-K）与季报（10-Q）的同比变化，尤其是管理层讨论与风险章节中的负面变化，对股票收益有较强的预测力。

## 拆成 ML 语言

- 任务 T：预测未来收益（回归）或超额收益方向（分类）；
- 经验 E：历年文本 + 对应收益；
- 度量 P：样本外 $R^2$、IC、多空组合夏普；
- 陷阱：文本发布日期与交易时点的时间泄漏（93 页）。

这个例子说明：特征工程常常比模型选择更重要——把" 文本变化" 变成可计算特征，是金融 ML 的主要工作量。

来源：案例据统计文本分析与资产定价文献的通行做法；数据与结论为文献综述表述，本讲不含真实数据。

# 商业应用 Applications in Business

## 七类场景

- 社交媒体特征提取
- 商品推荐
- 图像识别
- 情感分析
- 购买行为预测
- 客户满意度提升
- 营销优化与垃圾信息识别

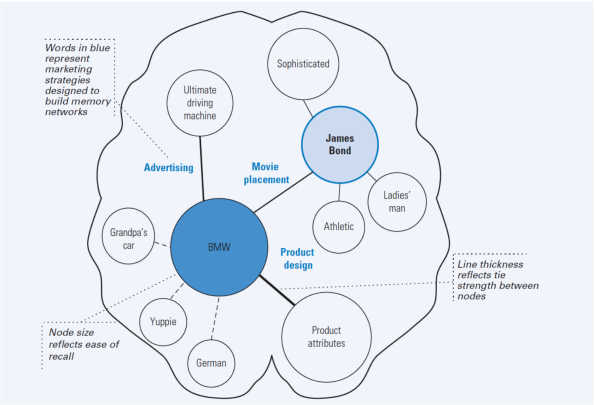
## 每一类都要问的三个问题

1. 标签从哪来？（购买记录、人工标注、还是代理指标）
2. 评估口径是什么？（点击率、转化率、还是长期价值）
3. 上线代价多大？（延迟、算力、误判成本）

推荐系统的典型陷阱：用曝光数据训练、用全量数据评估，忽略"没被推荐过的商品永远没机会"。

来源：场景清单据商业数据分析教材的通行分类；三个问题与陷阱为本讲补充。

# 案例：营销与品牌场景里的分析问题 A Marketing Case




品牌与营销是机器学习最常见的商业落点之一：分群、定位、投放、复购预测。但要提醒一句：图上的框架是" 业务逻辑"，不是" 模型"——模型只能回答" 哪一类客户响应更高"，回答不了" 品牌应该是什么"。判断仍然在人。

课堂案例素材（品牌策略示意图）；文字解读为本讲补充。

# 预测任务的三种形态 Three Data Shapes

| 形态 | 典型数据 | 关键注意点 |
| --- | --- | --- |
| **表格回归** | 房价、销量、违约损失 | 先做线性基线；检查非线性与交互；注意目标泄漏 |
| **表格分类** | 欺诈、流失、违约 | 类别不平衡、阈值与代价、概率校准 |
| **非结构化** | 文本、图像、音频、序列 | 先做特征表示（嵌入）；时间切分；标注成本高 |

经验法则：表格数据先试树与集成（梯度提升仍是强基线）；非结构化数据先试预训练表示 + 简单分类头，再考虑端到端微调（帧 86）。

Right tool for the data shape.

来源：形态划分与经验法则据表格数据与深度学习的实证对比研究整理。

# 核心动作：找模式，用模式 Find the Pattern, Apply the Pattern

## 核心原则

机器学习的目标是从大量数据中提取知识：通过从已知样例中学习，确定数据中的内在结构，从而在没有人工干预的情况下生成结构或预测。一种初级的理解方式是："找模式，用模式。"

## "无人工干预"的边界

- 人仍然决定：任务、度量、特征、阈值、部署方式；
- 模型只负责在给定的假设空间内搜索参数；
- 因此"自动化"不等于"无需判断"——判断被前移到了设计与评估阶段。

来源："找模式、用模式"的表述据机器学习基础教材；边界讨论为本讲补充（与帧 20 的自省呼应）。

# 监督学习 Supervised Learning

## 关键概念

- 输入 $X$（特征）与输出 $Y$（标签）成对出现；
- 学习的目标是推断从输入到输出的映射；
- 用训练数据训练，再对新输入预测输出。

## 两个必须区分的词

- 样本内（in-sample）：训练集上的表现，只是"记住"的程度；
- 样本外（out-of-sample）：新数据上的表现，才决定模型有没有用。

## 第一个基线：多元回归

回归模型用成对的 $(X, y)$ 估计参数，刻画 $y$ 与 $X$ 的关系；估计出的参数可用于对新的 $X$ 预测；预测值与真实值的差异用来评估样本外表现。

来源：监督学习的定义与多元回归示例据金融机器学习教材的基础章节。

# 无监督学习 Unsupervised Learning

## 核心说明

无监督学习不使用带标签的数据：我们只有输入，没有目标。算法要在数据内部发现结构。正因为如此，无监督学习特别适合探索新数据——为专家提供对"太大或太复杂、无法目视"的数据集的洞察。

## 两类主要问题

- 降维：在保留观测间变异的前提下减少特征数量；目的包括可视化、去噪、压缩，以及为下游模型提供更少但更稳的输入；
- 聚类：把观测分成若干组，使同组内更相似；分组标准可以预设（如组数）也可以由数据决定。

资产管理中的用法：按财报特征把公司分成经验分组，而不是沿用行业或国别这类传统分组。

来源：说明与两类问题的表述据金融机器学习教材；应用示例为该教材中的资产管理案例。

# 深度学习与强化学习 Deep Learning and Reinforcement Learning

## 深度学习

用更复杂的算法处理高度复杂的任务：图像分类、人脸识别、语音识别、自然语言处理。

## 强化学习

计算机通过与自身或环境交互来学习（数据也由算法自己产生），没有逐步的标签与即时反馈。

## 共同基础：神经网络

深度学习与强化学习都建立在神经网络之上：NN 是非常灵活的 ML 算法，已被成功用于非线性与特征交互突出的任务。除分类与回归之外，NN 也是深度学习与强化学习的基础，而这两者既可以是监督的，也可以是无监督的。

来源：表述据金融机器学习教材对神经网络、深度学习与强化学习的综述。

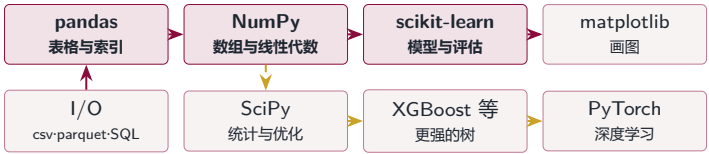
# 监督与无监督的对照 Supervised versus Unsupervised

| 维度 | 监督学习 | 无监督学习 |
|---|---|---|
| **前提假设** | 独立观测、环境稳定 | 独立观测、结构可由相似性定义 |
| **典型任务** | 回归 $E[Y \mid X = x]$、分类 $\Pr(Y = y \mid X = x)$ | 发现相似的组、压缩表示 |
| **数据类型** | 特征 + 标签 | 只有特征（图像、文档、个人上网行为等） |
| **度量** | 样本外误差、AUC、召回率 | 轮廓系数、稳定性、业务可解释性 |
| **跨学科差异** | 使用方法差异较小 | 使用方法差异较小 |

注意最上面一行："环境稳定" 是监督学习的隐含前提。分布一旦漂移，样本外表现就会崩塌——这正是 MLOps 要监控漂移的原因（帧 96）。

来源：对照表据金融机器学习教材中的监督/无监督对比页整理。

# 工具链全景：谁管什么 The Stack at a Glance




## 一句话分工

pandas 管数据、NumPy 管算、sklearn 管模型，matplotlib 管看，PyTorch 管" 当 sklearn 不够用时"。

## 为什么先学这三个

它们覆盖了八成以上的日常任务，而且API 设计互相呼应——学一个，另两个上手很快。

来源：分工据各库文档与本讲实践；生态图为本讲绘制。

# 任务 → 库 → 类：一张对照表 Task, Library, Class

| 任务 | 先看哪个库 | 首选类 | 备注 |
|---|---|---|---|
| **表格回归** | sklearn | `Ridge / RandomForestRegressor` | 先线性再看树 |
| **表格分类** | sklearn | `LogisticRegression / HistGradientBoosting` | 不平衡时调 class_weight |
| **要解释系数** | statsmodels | `OLS / Logit` | 有标准误与 p 值 |
| **强基线** | XGBoost/LightGBM | `LGBMClassifier` | 注意早停与正则 |
| **降维** | sklearn | `PCA / UMAP(第三方)` | 先标准化 |
| **聚类** | sklearn | `KMeans / HDBSCAN` | 先看轮廓系数 |
| **文本** | sklearn + Transformers | `TfidfVectorizer` / 预训练模型 | 先 TF-IDF 基线 |
| **时序** | statsmodels + sklearn | `ARIMA` / 特征化 +GBDT | 先做滞后特征 |

这张表是本讲的" 目录": 后面每一节都对应其中一行。

来源：对照表据各库文档与本讲实践整理。

# NumPy：把循环交给数组 NumPy

## 三个核心概念

ndarray（同类型、定形状的多维数组）；向量化（一次算一整片，不写 for）；广播（形状不同也能按规则对齐）。

## 为什么快

数据连续存放、循环在 C 里跑；比 Python 循环快几十到上千倍，这是后面所有模型能跑起来的前提。

| 操作 | 写法 | 典型用途 |
|---|---|---|
| **建数组** | `np.array, np.zeros, np.arange, np.linspace` | 造特征、造网格 |
| **形状** | `reshape, ravel, [:, None]` | 把向量变成列/矩阵 |
| **聚合** | `mean, std, sum(axis=0)` | 逐列统计 |
| **线性代数** | `@, np.linalg.solve, svd` | 回归的闭式解、PCA |
| **随机** | `default_rng, normal, permutation` | 可复现的采样 |

记住一条：看到 for 循环在数组上跑，先想能不能向量化。

来源：API 据 NumPy 官方文档；经验法则为本讲补充。

# pandas：把表格当一等公民 pandas

## 三个对象

Series（带索引的一列）、DataFrame（带索引的二维表）、Index（对齐用的标签）。

## 四个高频动作

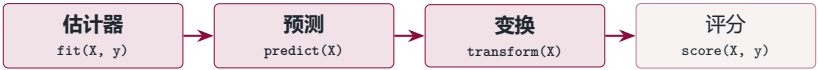
筛选（布尔索引）、分组（groupby）、合并（merge/join）、整形（pivot/melt）。

| 任务 | 写法 | 坑 |
|---|---|---|
| **读数据** | `read_csv, read_parquet` | 编码、千分位、日期列 |
| **选行列** | `loc, iloc` | loc 含右端、iloc 不含 |
| **分组统计** | `groupby(...).agg(...)` | 分组键含缺失会被丢掉 |
| **连接** | `merge(on=..., how=...)` | 一对多会悄悄放大行数 |
| **时序** | `to_datetime, resample` | 时区与频率对齐 |

pandas 的最大价值不是" 能读表"，而是索引对齐：两个表按标签自动对上，比手工拼数组安全得多。

来源：API 据 pandas 官方文档；坑为本讲实践总结。

# scikit-learn：所有模型同一张脸 One API




## 统一的四个方法

`fit` 学参数、`predict` 出预测、`transform` 改数据、`score` 打分。换模型只换一行 import。

## 三个"装配件"

Pipeline（把预处理和模型串起来）、ColumnTransformer（不同列不同处理）、GridSearchCV（连同交叉验证一起调参）。

为什么这个设计重要：它让"比较模型"变成可自动化的实验——同一个循环里换 6 个模型、3 组参数，跑完直接出表。

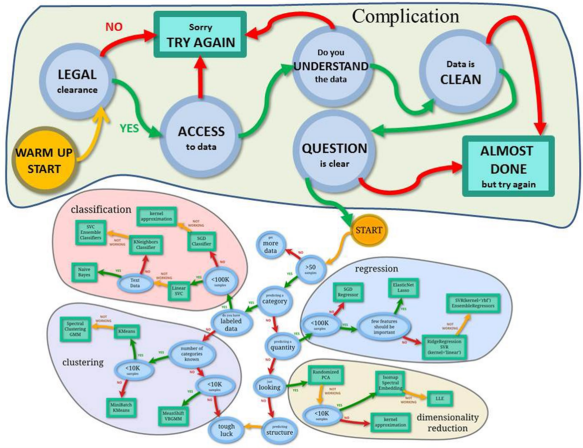
来源：API 设计据 scikit-learn 文档（Buitinck et al., 2013）；解读为本讲补充。

# 【代码】三库协同：一行流水线

```python
import numpy as np                                  # arrays and linear algebra
import pandas as pd                                 # tabular data
from sklearn.pipeline import Pipeline               # glue preprocessing and model
from sklearn.preprocessing import StandardScaler    # standardise the features
from sklearn.linear_model import LogisticRegression # the model
df = pd.read_csv("orders.csv", parse_dates=["date"]) # pandas reads with a date column
df["margin"] = df["revenue"] - df["cost"]           # a new column, vectorised
X = df[["margin", "discount", "tenure"]].to_numpy() # pandas -> numpy, one call
y = (df["churn"] == "yes").astype(int).to_numpy()   # target as 0/1
pipe = Pipeline([("scale", StandardScaler()),       # step 1: standardise
                 ("clf", LogisticRegression(max_iter=1000))]) # step 2: the classifier
pipe.fit(X[:800], y[:800])                          # fit on the first 800 rows
print("accuracy on hold-out: %.3f" % pipe.score(X[800:], y[800:])) # score on the rest
```

注意三件事：日期列要 parse、目标要变成 0/1、标准化必须放进 Pipeline（否则会泄漏）。完整程序见 `website/code/lecture05/`。

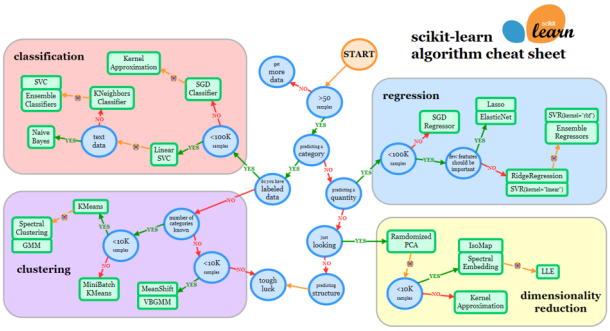
# 【样本 + 可视化】scikit-learn 估计器地图（一）




官方那张著名的" 该用哪个估计器" 流程图。读法：从左上角开始，先问" 有标签吗"，再问" 是分类还是回归"，最后问" 样本量/是否线性"。它不是标准答案，而是一张排除表——帮你把候选缩到 2–3 个。

图片来源：scikit-learn 官方文档 (estimator map)；本页仅作教学示意。

# 【样本 + 可视化】估计器地图（二）与读法说明




## 三步读出候选

(1) 有无标签 → 监督/无监督；(2) 预测连续还是类别 → 回归/分类；(3) 数据规模与线性程度 → 选族。

## 三条经验补充

先线性基线再上复杂模型；样本少于 1 万先试树/集成；文本先做向量化再谈模型。

来源：图片来源：scikit-learn 官方文档；三步读法与本讲经验为课程补充。

# 其它重要库：什么时候需要它们 Beyond the Big Three

| 库 | 擅长 | 你会在什么时候用它 |
|---|---|---|
| **matplotlib / seaborn** | 静态图与统计图 | 每次探索与报告 |
| **statsmodels** | 带标准误的计量模型 | 要解释系数、要 p 值与置信区间 |
| **XGBoost / LightGBM** | 表格数据的强基线 | 结构化数据的默认首选 |
| **imbalanced-learn** | 采样与代价敏感 | 违约、欺诈这类稀有事件 |
| **SHAP / 部分依赖** | 事后解释 | 要给业务解释" 为什么" |
| **PyTorch / Transformers** | 深度学习与预训练模型 | 文本、图像，或 sklearn 不够时 |

一条判断：能用 sklearn + 梯度提升解决的，就不要上深度学习。深度学习的收益在非结构化数据和大规模，不在" 更高级"。

来源：库分工据各项目文档；判断为本讲经验。

# 库的边界与四个坑 Four Traps in Practice

1. 版本漂移：0.x 到 1.x 的行为变了（默认参数、报错信息）；
2. 随机种子：不固定就复现不了，‘random_state‘ 要贯穿始终；
3. 稀疏与内存：文本向量化后是稀疏矩阵，别转成密集数组；
4. 静默的类型转换：object 列会悄悄拖慢一切，先 `astype`。

这四条合起来就是一句：工程问题常常比模型问题更致命。本讲后面每张" 样本页" 背后，都有一份被固定过的环境与种子。

来源：四个坑据实践总结；对应本讲程序的 `requirements.txt` 与种子设置。

# 预测问题与因果问题 Prediction versus Causation

## 预测问题

目标：在只看得到 $X$ 的新数据上，估计并最小化均方误差。不需要知道"为什么"，只需要预测得准：可以做样本外检验，可以比较不同模型。

## 因果问题

目标：估计干预的效果（"如果价格提高 5%，销量会怎样"）。需要识别策略（实验、工具变量、断点等），而且无法用留出集检验——因为反事实观测不到（衔接 L12）。

## 很多问题可以分解为两部分

- 用现成的 ML 处理预测部分（例如先估出需求函数中的 nuisance 部分）；
- 用计量方法处理因果部分（把估计量对准感兴趣的参数）；
- 两部分共享同一条纪律：数据划分、验证、记录。

来源：预测与因果的分工据经济学与机器学习交叉领域的综述（Athey、Mullainathan 等）。

# 两种自省：我们说的与我们做的 Two Kinds of Self-Awareness

## 机器学习侧

- 说：只用预测说话，把人从选择中解放出来；
- 做：用预测模型并忽略其他考虑——因果、均衡与反馈效应；
- 心里担心可解释性、可靠性、稳健性、适应性，却缺乏让算法直接优化这些目标的渠道。

## 计量经济学侧

- 说：报告因果效应与标准误，再加几个稳健性设定；
- 做：跑很多函数形式、把研究助理跑的几百个回归里" 挑几个有代表性的"；
- 按" 只跑了一个模型" 的方式报告标准误，隐含多重比较问题。

两侧的毛病其实同源：选择过程没有被记录。共同的解药是预注册 + 留出验证 + 完整报告——这也正是本讲反复强调" 评估协议要落盘" 的原因。

来源：两侧的" 说的与做的" 对照据计量经济学与机器学习的方法论评述整理。

# 机器学习与传统计量经济学的差别 ML versus Econometrics

| 维度 | 传统计量经济学 | 机器学习 |
|---|---|---|
| **数据规模** | 关注 $N \gg P$（样本远多于变量） | 关注 $P$ 很大、甚至 $P > N$ |
| **样本内误差** | 可近似样本外误差；OLS 是 BLUE，不必引入偏差 | 样本内误差会严重低估样本外误差，必须留出验证 |
| **共线性** | 方差膨胀、系数不稳 | 正则化与集成可以当作" 降方差" 的手段 |
| **核心目标** | 估计因果效应与识别 | 提高样本外预测表现 |
| **检验方式** | 标准误、识别假设的可信度 | 留出集与交叉验证（无法检验识别假设） |

一句话总结：目标不同，评价方式就不同。预测任务用样本外误差当裁判；因果任务必须靠设计，而不是靠" 预测得准"。

来源：对照据计量经济学与机器学习比较的方法论文献；结论为通用共识。

# 当变量比样本还多 When P Exceeds N

## 会发生什么

- 样本内误差可以做到接近零（甚至完全为零）；
- OLS 在 $P > N$ 时根本没有唯一解；
- 只要允许模型足够复杂，它就能" 记住" 训练数据，包括其中的噪声；
- 于是样本外表现可能比简单模型更糟。

## 本讲的实测例子

房价样本 1,200 行、7 个原始特征，做三次多项式展开后变成 164 个特征，而训练集只有 900 行——此时不加惩罚的线性模型已经不稳，Ridge/Lasso 立刻变得重要（帧 39、51）。

三条应对：正则化（惩罚复杂度）、降维（用更少的方向概括变异，帧 64）、留出验证（用样本外误差做裁判）。

来源：高维下的过拟合与 OLS 失效为统计学习的基本结论；数值例子来自本讲程序 l5_05。

# 给计量经济学的几点启示 Lessons for Empirical Work

## 方法论启示

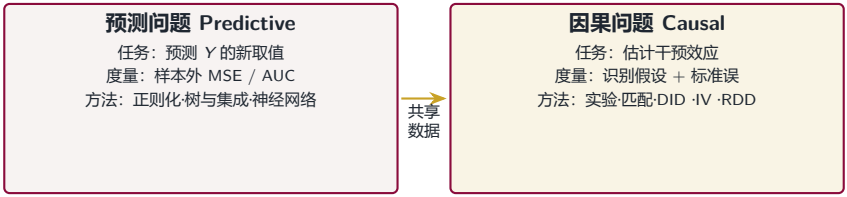
1. 很多问题可以分解为预测部分与因果部分；
2. 预测部分可以直接用现成 ML；
3. 用数据驱动的方式做模型选择，但准则要服从研究目标；
4. 把注意力放在感兴趣的参数上；
5. 在能评估表现的地方使用数据驱动选择，同时保留推断能力。

## 验证的启示

- ML 永远有测试集；计量研究也可以设计" 更难的测试"：例如在价格异常的日子评估预测，或比较价格变动前后的购买变化；
- 技术公司天然拥有大量 A/B 测试与算法变更，它们是研究反馈与均衡效应的天然实验场；
- 结构模型的替代计算工具：随机梯度下降、变分推断。

来源：启示据统计学与经济学交叉的方法论文章整理；引用示例为该领域的代表性研究。

# 一张图看清分工 Who Does What

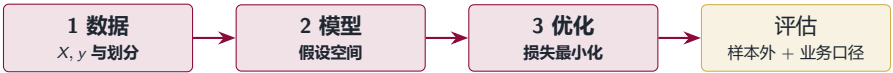



两者共享同一份数据、同一套验证纪律、同一份文档；区别在于" 能不能用留出集检验"：预测可以，因果不行。

Two goals, one dataset, different referees.

来源：分工图为据预测与因果文献整理的示意；具体识别策略在 L12 展开。

# 机器学习 1-2-3 Machine Learning in Three Steps




## 每一步的常见错误

- 数据：泄漏、划分不可比、样本量不足；
- 模型：假设空间与问题不匹配（线性模型做非线性任务）；
- 优化：不收敛、学习率不当、把调参当建模；
- 评估：用测试集调参、只看准确率、忽略代价。

## 顺序不能颠倒

先定度量，再定划分，最后才选模型。反过来做（先挑模型再想怎么评）几乎必然得到乐观结论——这也是本讲把评估放在算法族之前的原因。

来源：1-2-3 框架沿用课程 L1 的表述；常见错误据本项目实践与教科书整理。

# 训练、验证与测试 Three Splits

## 三个不重叠的样本

- 训练集：用来训练模型（样本内）；
- 验证集：用来比较方案与调超参数；
- 测试集：只用一次，用来确认模型在新数据上的表现。

常用比例 60/20/20；样本很小时可用交叉验证替代验证集。

## 本讲的划分（信用卡样本）

3,000 行、正类比例 26.7%，按 60/20/20 分层切分：训练 1,800 行、验证 600 行、测试 600 行；分层保证三份数据的正类比例一致。

两种必须改变划分方式的场景：(1) 时间序列（按时间切分，不能随机打乱）；(2) 分组数据（同一客户/同一门店的记录不能跨集合，否则是分组泄漏）。

来源：三划分与比例据统计学习通行做法；样本信息来自本讲程序 l5_01 的实测输出。

# 交叉验证 Cross-Validation

## 基本做法

- 把训练数据分成 $k$ 折（常用 5 或 10）；
- 每次用 $k-1$ 折训练、留 1 折评估，轮换 $k$ 次；
- 报告 $k$ 个得分的均值与标准差；
- 调参时用嵌套交叉验证，避免"调参泄漏"。

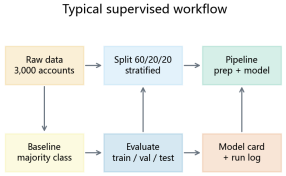
## 五种划分器

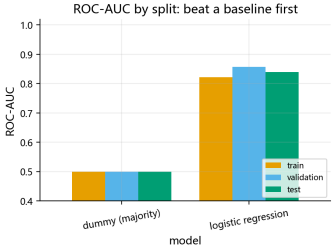
| 划分器 | 何时使用 |
|---|---|
| KFold | 回归等无类别不平衡问题 |
| StratifiedKFold | 分类，保持各折正类比例 |
| GroupKFold | 分组数据(同一实体不跨折) |
| TimeSeriesSplit | 时间序列，只用过去预测未来 |
| LeaveOneOut | 小样本，计算代价高 |

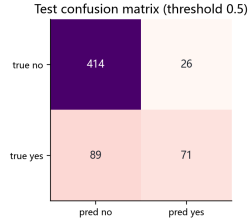
纪律：测试集绝不参与调参。任何"在测试集上试了几次"的模型，其报告分数都已经是乐观估计——这与帧 30 的泄漏实验是同一类错误。

来源：交叉验证做法与划分器选择据 scikit-learn 用户指南与统计学习实践整理。

# 【样本 + 可视化】端到端流水线与基线 Pipeline and Baseline




3,000 笔账户、正类 26.7%：多数类基线准确率 73.3%，但 ROC-AUC 只有 0.500——只看准确率会被基线骗过；逻辑回归样本外 AUC 0.840、准确率 80.8%，才算" 有信息"。

程序 `15_01_pipeline_baseline.py` (每行英文注释) ·样本 `15_credit_default.csv` ·结果 `15_baseline_scores.csv`

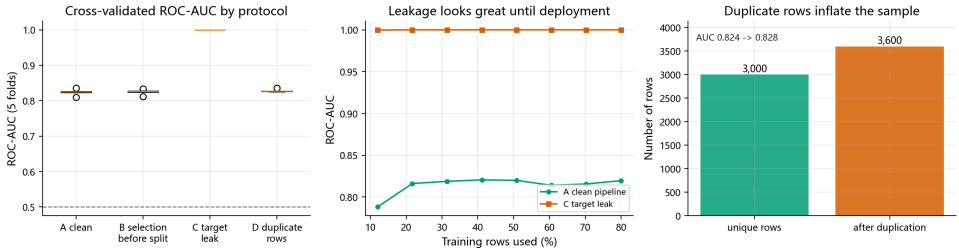
# 【代码】划分、流水线与交叉验证 Splitting and Cross-Validation

```python
X_train, X_tmp, y_train, y_tmp = train_test_split(    # first split: training versus the rest
    X, y, test_size=0.4, random_state=42, stratify=y) # stratify keeps the default rate identical
X_val, X_test, y_val, y_test = train_test_split(      # second split: validation versus test
    X_tmp, y_tmp, test_size=0.5, random_state=42, stratify=y_tmp) # both halves keep the same rate
prep = ColumnTransformer(                             # route each column type to its transformer
    transformers=[("num", StandardScaler(), num_cols), # numeric columns are standardised
                  ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols)]) # categories become dummies
def make_pipe(model):                                 # build a fresh pipeline for an estimator
    return Pipeline(steps=[("prep", prep), ("model", model)]) # preprocessing then estimator
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42) # five stratified folds, fixed seed
scores_clean = cross_val_score(clean, X, y, cv=cv, scoring="roc_auc") # honest cross-validated AUC
scores_target = cross_val_score(model_only, X_target, y, cv=cv, scoring="roc_auc") # leaked column included
```

要点：预处理必须放进流水线，否则每一折的信息会互相污染；交叉验证只报告均值与标准差，不要只报一个最好看的折。

完整程序 `15_02_cv_and_leakage.py`。

# 【样本 + 可视化】四种协议与四个分数 Four Protocols




同一模型：诚实流水线 0.824、先选特征再切分 0.825、目标泄漏 1.000（把结果当特征）、重复行 0.828；诚实留出集给出 0.865——分数只反映协议，不反映模型。

程序 `15_02_cv_and_leakage.py` ·结果 `15_cv_results.csv` ·泄漏清单见帧 93

# 过拟合 Overfitting

## 核心定义

一个 ML 算法如果把训练数据拟合得太好，通常无法在新数据上预测得好：也就是说，它泛化得不好。泛化好的模型是在样本外仍保持解释力的模型。

## 过拟合是怎么发生的

过拟合模型把训练数据中的噪声与随机波动也学进了关系里；这些波动不存在于任何新数据中，因此会损害泛化能力。所以评估的重点是新数据上的预测误差，而不是训练数据上的拟合优度。

识别信号：训练误差很低、验证误差明显更高，且两者差距随复杂度单调扩大。

来源：过拟合与泛化的表述据金融机器学习教材的评估章节。

# 欠拟合 Underfitting

## 核心定义

欠拟合指的是模型无法准确捕捉输入与输出之间的关系，从而在训练集与未见数据上都产生较高的误差。

## 常见原因与对策

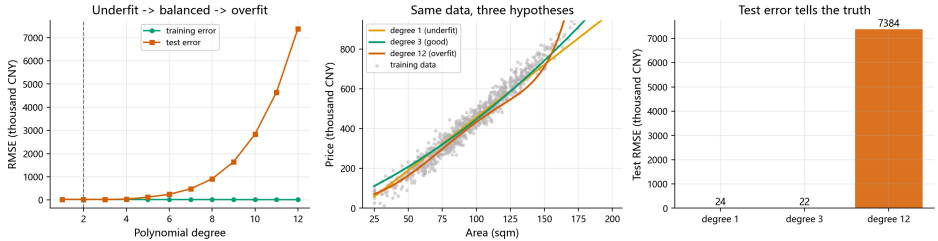
- 模型过于简单（线性模型拟合弯曲关系）；
- 特征不足或特征表达不好（缺少交互项）；
- 正则化过强（惩罚把信号也压掉了）；
- 训练不充分（迭代次数太少、学习率太小）。

对策：增加特征或提高模型容量，并重新检查学习曲线（帧 36）。

与过拟合一样，欠拟合的模型也无法用于分类或预测；泛化能力最终才是让机器学习算法每天可用的原因。

来源：欠拟合定义与原因据金融机器学习教材；对策为据学习曲线诊断的实践整理。

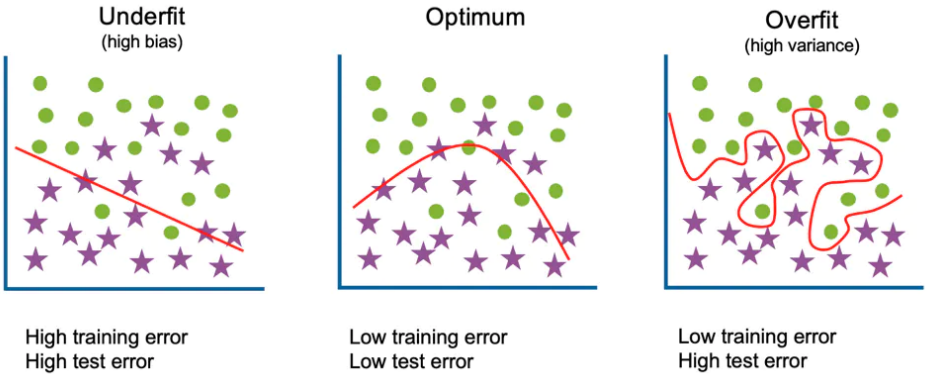
# 【样本 + 可视化】同一数据、三种假设 Three Hypotheses




房价样本（1,200 行，均价 42.3 万）：多项式次数 1（欠拟合）、2（本样本最佳）、12（过拟合）；左图显示训练误差单调下降而测试误差先降后升——这就是"复杂度旋钮"的代价。

程序 `l5_03_overfitting_curves.py` · 样本 `l5_housing.csv` · 扫描表 `l5_complexity_table.csv`

# 【样本 + 可视化】欠拟合与过拟合长什么样 Underfit versus Overfit




同一条真实关系，三种拟合：左 = 欠拟合（太简单的假设，偏差大）、中 = 恰当、右 = 过拟合（穿过每个点，方差大）。判据不是" 线弯不弯"，而是" 换一批数据，它还稳不稳"。

课堂素材图：配合帧 30–32 的三类假设与帧 35 的学习曲线一起看。

# 【代码】复杂度扫描与学习曲线 Complexity Sweep

```python
for degree in range(1, 13):                          # degrees one to twelve
    pipe = Pipeline(steps=[("poly", PolynomialFeatures(degree=degree, include_bias=False)), # expansion
                           ("scale", StandardScaler()),   # standardise the expanded features
                           ("model", Ridge(alpha=1.0))])  # ridge keeps high degrees stable
    pipe.fit(X_train, y_train)                       # fit on the training split
    train_rmse = np.sqrt(mean_squared_error(y_train, pipe.predict(X_train))) # in-sample error
    test_rmse = np.sqrt(mean_squared_error(y_test, pipe.predict(X_test))) # out-of-sample error
def curve_for(degree, ridge_alpha):                  # helper returning train/test curves
    sizes, train_err, test_err = learning_curve(     # compute the learning curve
        pipe, X_train, y_train, cv=5, shuffle=True, random_state=42, # five folds, shuffled
        train_sizes=np.linspace(0.1, 1.0, 8), scoring="neg_root_mean_squared_error") # RMSE per size
    return sizes, -train_err.mean(axis=1), -test_err.mean(axis=1) # flip the sign back to positive RMSE
```

要点：一次扫描得到" 复杂度—误差" 曲线，再用 `learning_curve` 看样本量的作用；两条曲线一起看，才能区分" 缺样本" 与" 缺约束"。完整程序 `15_03_overfitting_curves.py`。

# 三类误差：偏差、方差与基误差 Three Sources of Error

## 三类误差

1. 偏差误差：模型拟合训练数据的程度。假设错误会产生高偏差、近似能力差，导致欠拟合与高样本内误差；
2. 方差误差：模型结果对新数据的敏感程度。不稳定的模型会捡起噪声、产生高方差，导致过拟合与高样本外误差；
3. 基误差：数据本身的随机性带来的误差（不可消除）。

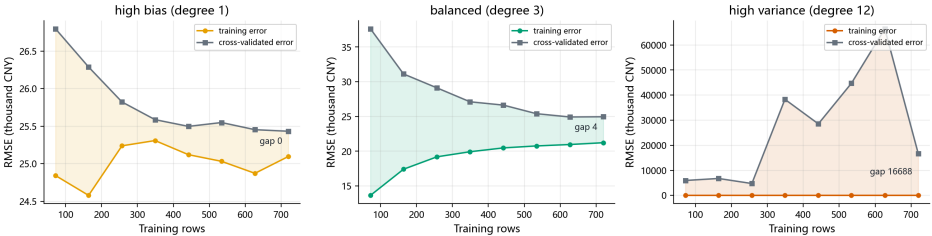
来源：三类误差的表述据金融机器学习教材；方差-偏差分解为统计学习标准结论。

## 分解式

$$\mathbb{E}\left[(y-\hat{f}(x))^2\right] = \underbrace{\mathrm{Bias}^2}_{\text{模型误差}} + \underbrace{\mathrm{Var}}_{\text{估计误差}} + \underbrace{\sigma^2}_{\text{噪声}}$$

含义：最小化均方误差必然要接受一些偏差；" 更复杂的模型一定更好" 是错的。

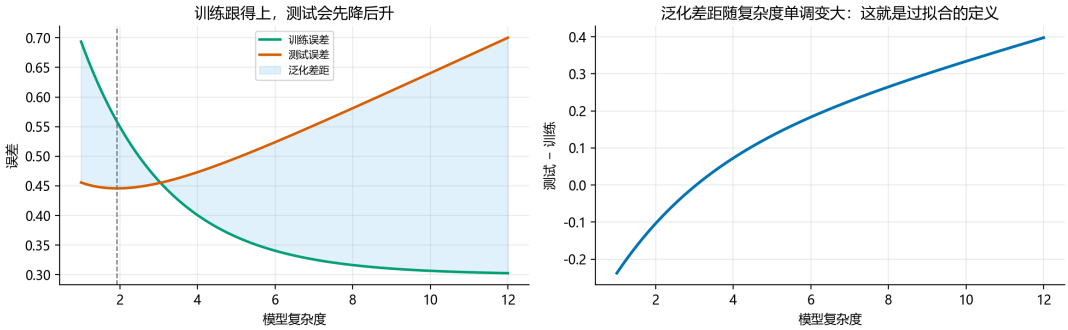
# 【样本 + 可视化】学习曲线诊断 Diagnosing with Learning Curves




三种模型的训练误差（实线）与交叉验证误差（虚线）随样本量变化：高偏差时两线都高且靠得近；高方差时训练误差极低而验证误差迟迟不降、两条线之间留着大缺口——加数据比换模型更有效。

程序 `15_03_overfitting_curves.py` 曲线用 `learning_curve` 5 折计算，误差为 RMSE（千元）

# 【样本 + 可视化】泛化差距：过拟合的定量定义




左：训练误差一路下降到接近 0，测试误差先降后升，两条线之间的面积就是泛化差距；右：差距随复杂度单调变大——所以" 训练集上 99%" 从来不是成就。最优复杂度在测试曲线的最低点，不在训练曲线的最低点。

程序 `tools_figs_l5.py` ·图 `fig_generalization_gap.png`

# 学习曲线的两种含义 Two Meanings of a Learning Curve

## 机器学习的学习曲线

横轴是训练样本量，纵轴是准确率或误差；稳健的模型表现为：随着训练数据增加，样本外准确率上升，训练误差与验证误差相互靠拢，并趋向一个理想误差（或基误差）。

## 人类的学习曲线

同样叫" 学习曲线"，但横轴是经验或练习量、纵轴是熟练度：做得越多、组织与行业做得越久，任务完成得越好。

同名异义提醒：读文献时先看横轴是什么。本课程里的" 学习曲线" 若无特别说明，都指样本量—误差曲线。

来源：两种含义的对照据金融机器学习教材与组织学习文献中的同名概念。

# 防止过拟合的两条原则 Two Principles against Overfitting

## 原则一：限制复杂度（奥卡姆剃刀）

最简单的解往往是正确的解。在监督学习中，这意味着限制特征数量，并惩罚过于复杂或过于灵活的算法——约束它们只保留能降低样本外误差的参数。

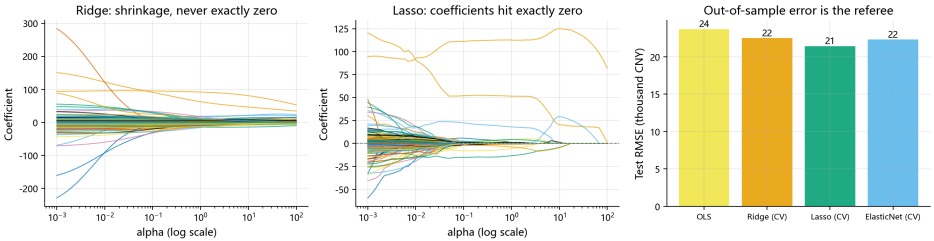
## 原则二：避免抽样偏差

好的实验设计有代表性样本与控制组，且二者应来自同一理论分布。监督学习中的做法是谨慎地把数据分成三份：训练样本（标签已知）、验证样本（比较与调参）、测试样本（确认预测能力）。目标是让模型部署到同一领域的全新数据上。

两条原则对应两类技术：惩罚（Ridge/Lasso、剪枝、早停）与采样（交叉验证、分层、时间切分、分组切分）。

来源：两条原则与奥卡姆剃刀的表述据金融机器学习教材的评估章节。

# 【样本 + 可视化】正则化路径：收缩与稀疏 Ridge vs Lasso




三次展开后有 164 个特征（训练行只有 900）：Ridge 让系数整体收缩但不归零，Lasso 让系数逐个归零；右图的样本外误差显示惩罚比不惩罚更准，但三种惩罚之间差异不大——选型要看稀疏性是否对你有用。

程序 `15_05_regularized_regression.py` ·样本 `15_housing.csv` ·结果 `15_regression_scores.csv`

# 泛化误差与归纳偏置 Generalisation Error and Inductive Bias

## 监督学习的目标

用输入对象（预测变量向量）与期望输出（人工标注的监督信号）训练模型；训练过程构建一个把新数据映射到期望输出的函数。理想情况下，算法能对未见实例给出正确输出。

## 归纳偏置

这要求学习算法能以" 合理" 的方式从训练数据泛化到未见情形——这个" 合理" 来自算法的归纳偏置（它偏好什么样的解）。算法的这一统计性质通过泛化误差来度量。

例子：线性模型的偏好是" 平滑的线性关系"；KNN 的偏好是" 局部相似"；树模型的偏好是" 轴平行的矩形划分"。偏好与数据形态不匹配，就会欠拟合。

来源：监督学习定义与泛化误差的表述据机器学习基础教材；归纳偏置示例为本讲补充。

# 回归任务的评估指标 Regression Metrics

| 指标 | 定义与特点 | 何时使用 |
|---|---|---|
| **MSE** | 均方误差，对大误差惩罚重 | 作为优化目标（可导、便于分解） |
| **RMSE** | 与目标同量纲（本讲房价单位：千元） | 向业务方汇报误差大小 |
| **MAE** | 绝对误差均值，对离群点更稳健 | 误差分布厚尾时 |
| $R^2$ | 解释的方差比例，与基线比较 | 需要"相对提升"的说法时 |
| **MAPE** | 百分比误差，跨量纲可比 | 目标值远离零且量纲差异大时 |

纪律：报误差时必须给出基线（例如"用均值预测"的 RMSE）；并且同一份报告里不要混用不同指标做模型对比。

来源：指标定义据统计学习与 scikit-learn 文档；汇报纪律为本讲补充。

# 分类任务的评估指标 Classification Metrics

## 从混淆矩阵出发

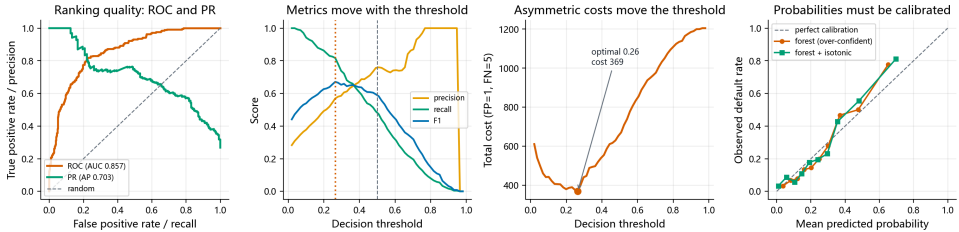
- 准确率：正确比例——类别不平衡时会被基线骗过；
- 精确率：预测为正的里面有多少真的正；
- 召回率：真的正里面有多少被抓到；
- F1：精确率与召回率的调和平均；
- ROC-AUC：跨阈值的排序能力；
- PR-AUC：正类稀少时更敏感。

来源：指标定义据机器学习评估通行做法；数值来自本讲程序 l5_01 与 l5_04 的实测输出。

## 本讲样本的实测

信用卡样本正类 26.7%：多数类基线准确率 73.3%、AUC 0.500；逻辑回归测试集 AUC 0.840。在 0.5 阈值下正类召回率只有 0.444——阈值没调好，模型再好也没用（帧 43）。

# 【样本 + 可视化】指标、阈值与校准 Metrics, Thresholds




测试集 ROC-AUC 0.857、平均精度 0.704；在" 漏判代价是误报 5 倍" 的假设下，最优阈值是 0.26 而不是 0.5（总代价 369）；随机森林概率偏自信（Brier 0.146），等渗校准后降到 0.143，逻辑回归本身已达 0.130。

程序 `l5_04_metrics_thresholds.py` ·阈值表 `l5_threshold_table.csv` ·样本 `l5_credit_default.csv`

# 代价敏感与阈值选择 Cost-Sensitive Decisions

## 把业务代价写进去

- 误报（FP）代价：人工复核成本、客户打扰；
- 漏判（FN）代价：坏账损失、欺诈金额；
- 期望总代价 $= \text{cost}_{FP} \cdot FP + \text{cost}_{FN} \cdot FN$；
- 在阈值网格上最小化期望代价，得到业务最优阈值。

## 三条实务建议

1. 阈值不进模型、进决策层：模型输出概率，阈值由业务定；
2. 阈值变化要说清召回率/精确率的交换；
3. 代价不对称时，用代价曲线而不是 AUC 做最终选择。

本讲样本：把漏判代价设为误报的 5 倍后，最优阈值从 0.5 降到 0.26——同样的模型，决策规则一变，业务结果就变。

来源：代价敏感决策据信用风险与欺诈检测的业务实践；数值来自本讲程序 l5_04。

# 【代码】指标、阈值与概率校准 Metrics, Thresholds, Calibration

```python
fpr, tpr, _ = roc_curve(y_test, proba)                # ROC curve points
roc_auc = auc(fpr, tpr)                               # area under the ROC curve
prec, rec, thr_pr = precision_recall_curve(y_test, proba) # precision-recall curve points
ap = average_precision_score(y_test, proba)           # average precision, the PR summary
for t in thresholds:                                  # loop over the threshold grid
    pred = (proba >= t).astype(int)                   # hard labels at this threshold
    rows.append({"threshold": round(float(t), 3),     # threshold value
                 "precision": round(precision_score(y_test, pred, zero_division=0), 4), # precision
                 "recall": round(recall_score(y_test, pred, zero_division=0), 4), # recall
                 "f1": round(f1_score(y_test, pred, zero_division=0), 4)}) # harmonic mean
costs = cost_curve(y_test, proba, thresholds, cost_fp=1.0, cost_fn=5.0) # FN costs five times an FP
best_cost_idx = int(np.argmin(costs))                 # threshold with the lowest total cost
calibrated = CalibratedClassifierCV(                  # wrap the forest with isotonic calibration
    Pipeline([("prep", prep), ("model", RandomForestClassifier(random_state=42))]), # base pipeline to calibrate
    method="isotonic", cv=5)                          # isotonic regression fitted with cross-validation
```

要点：曲线描出全部阈值，再用业务代价挑一个点；概率要校准后才可用于" 期望损失" 计算。完整程序 `15_04_metrics_thresholds.py`。

# 概率校准与不确定性 Calibration and Uncertainty

## 校准是什么意思

模型说"70% 会违约" 的时候，如果现实中确实约 70% 违约，概率就是校准的（reliability diagram 落在对角线上）。树与集成模型常常过度自信，需要等渗（isotonic）或 Platt 校准。

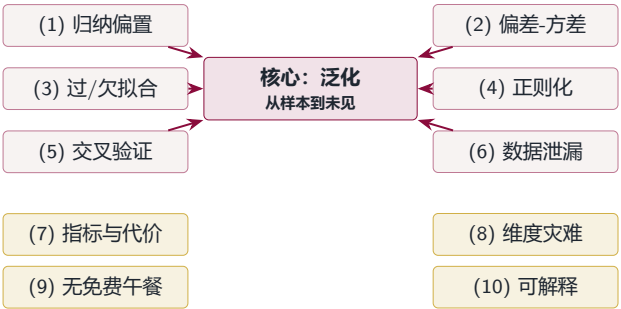
## 除了校准还要报什么

- 样本量与置信区间（L4 已讲过的自助法可直接复用）；
- 校准指标：Brier 分数、可靠性曲线；
- 决策的不确定性：阈值附近的样本占比；
- 分布漂移下的重校准计划（帧 96）。

一句话：概率用于决策，就必须可信；只报 AUC 不报校准，等于把" 排序能力" 当成了" 概率质量"。

来源：校准方法（isotonic、Platt）与 Brier 分数据 scikit-learn 文档与概率预测文献。

# 十个核心概念：一张图串起来 The Concept Map




前六个是" 模型为什么行/为什么不行"，后四个是" 怎么判断它真的行"。接下来逐条给数字。

来源：概念清单为本讲组织；下页起每帧两条。

# 概念 (1)(2)：归纳偏置与偏差-方差

## (1) 归纳偏置 Inductive Bias

模型自带假设：线性模型假设" 关系是直线"，树假设" 关系是分段常数"，KNN 假设" 近的相似"。没有偏置就无法泛化。

## (2) 偏差-方差 Bias-Variance

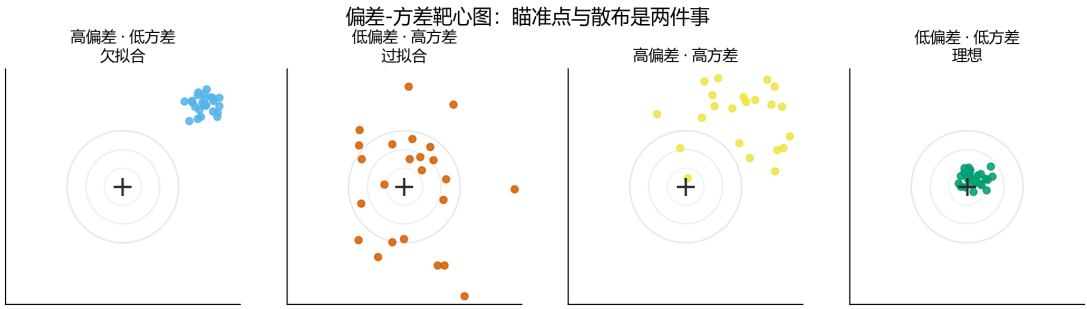
$$\mathbb{E}[(y-\hat{f})^2] = \text{偏差}^2 + \text{方差} + \text{噪声},$$

欠拟合 = 偏差大，过拟合 = 方差大。

这两个概念合起来说清一件事：你不是在" 找真模型"，而是在做一次偏差与方差的交易。下一帧的靶心图把这个交易画出来。

来源：偏差-方差分解据 Geman et al. (1992)；表述为本讲组织。

# 【样本 + 可视化】偏差-方差靶心图 The Target




四张靶：偏（瞄不准）与散（手抖）是两个独立的问题。左一欠拟合（偏而集中）、左二过拟合（准而散）、右下才是理想。调模型就是同时压低这两个方向。

程序 website/code/lecture05/tools_figs_l5.py ·图 fig_bias_variance_target.png

# 概念 (3)(4)：过拟合与正则化

## (3) 过拟合 Overfitting

训练误差继续降、测试误差回升——模型开始记忆噪声。诊断看两条曲线的分叉。

## (4) 正则化 Regularisation

在损失里加复杂度惩罚：L2 把系数往 0 拉 (Ridge)，L1 让系数变 0 (Lasso)。用一点偏差换很多方差。

| 手段 | 作用对象 | 典型强度 | 代价 |
|---|---|---|---|
| **L2 / Ridge** | 系数大小 | $\alpha$ 从 0 到 100 | 偏差上升 |
| **L1 / Lasso** | 稀疏性 | 同上 | 可能过稀疏 |
| **early stopping** | 训练轮数 | 验证误差最小点 | 需要验证集 |
| **剪枝 / 深度** | 树与网络 | 叶最小样本、层数 | 欠拟合风险 |

来源：正则化据 Hoerl & Kennard (1970) 与 Tibshirani (1996)；表格为本讲整理。

# 概念 (5)(6)：交叉验证与数据泄漏

## (5) 交叉验证 Cross-Validation

把数据切成 $k$ 折，轮流当验证集。它估计的是" 在新数据上的表现"，不是" 在训练数据上拟合得多好"。

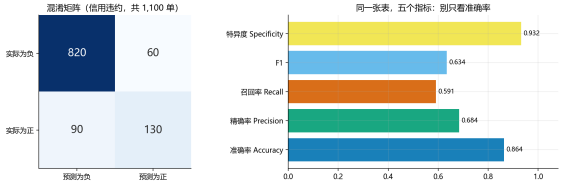
## (6) 数据泄漏 Leakage

训练时偷看了未来或答案：先用全样本标准化、用未来信息做特征、把目标编码进特征。症状：指标好得不真实。

这两条是一对：交叉验证给你一个诚实的分数，泄漏让这个分数变成谎言。本讲后面有专门的清单 (帧 90 的泄漏清单)。

来源：交叉验证据 Kohavi (1995)；泄漏分类据 Kaufman et al. (2012)。

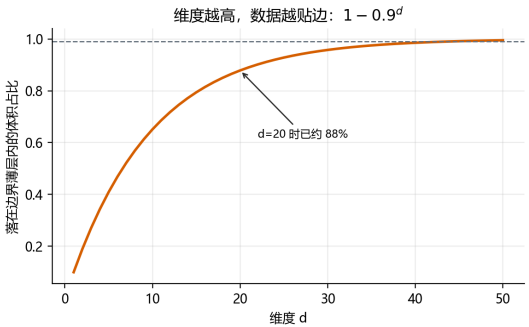
# 【样本 + 可视化】指标、维度灾难与泛化差距

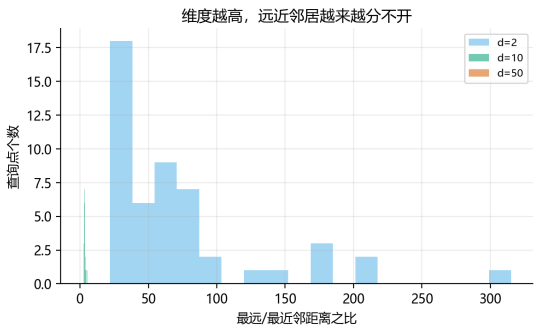



左：同一张混淆矩阵能算出准确率 0.864，但召回率只有 0.591——”看起来不错”和”真的管用”是两件事；
右：五个指标一起看，才知道模型偏在哪。

程序 `tools_figs_l5.py` ·图 `fig_confusion_metrics.png`（合成数据：1,100 单）

# 【样本 + 可视化】维度灾难：为什么”特征越多越好”是错的




左：$d = 20$ 时约 88% 的体积贴在边界薄层里——中心几乎没数据；右：维度越高，最近邻与最远邻的距离越来越接近，”近的相似”这个假设失效。这就是 KNN 在高维失效、也是降维存在的理由。

程序 `tools_figs_l5.py` ·图 `fig_curse_dimensionality.png`

# 十条速查：一页背完 Ten Concepts on One Page

| # | 概念 | 一句话 + 你该怎么用 |
|---|---|---|
| 1 | 归纳偏置 | 模型自带假设；选模型就是选假设 |
| 2 | 偏差-方差 | 欠拟合偏、过拟合散；交易而非消灭 |
| 3 | 过拟合 | 训练降、测试升；看两条曲线的分叉 |
| 4 | 正则化 | 加惩罚压复杂度；用偏差换方差 |
| 5 | 交叉验证 | 诚实估计泛化；别用训练分数自夸 |
| 6 | 数据泄漏 | 指标好得不真实；先查时间与全样本操作 |
| 7 | 指标与代价 | 准确率会骗人；用混淆矩阵与代价 |
| 8 | 维度灾难 | 高维下距离失效；降维或加正则 |
| 9 | 无免费午餐 | 没有万能算法；按数据形态选 |
| 10 | 可解释性 | 能解释才敢上线；模型卡是最低要求 |

这十条会在后面每一节里反复出现——看到它们时，回想这一页。

来源：十条概念为本讲组织；对应帧 2 的学习目标。

# 算法族地图 A Map of Supervised Algorithms

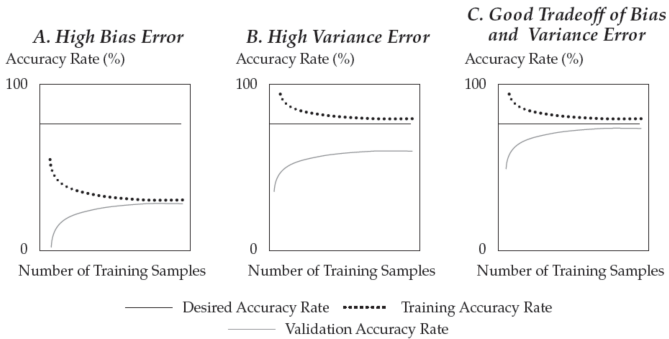
| 家族 | 代表方法 | 特点、代价与坑 |
|---|---|---|
| **线性模型** | 线性回归、逻辑回归、Ridge/Lasso | 快、可解释、可给系数置信；非线性关系下欠拟合 |
| **距离模型** | KNN | 无需训练、天然非线性；预测慢、对量纲与维度敏感 |
| **间隔模型** | SVM（线性/核） | 小样本高维表现好；调参敏感、概率需校准、大数据慢 |
| **树模型** | CART | 可解释、能处理混合类型；方差大、易过拟合 |
| **集成模型** | 随机森林、梯度提升 | 表格数据的强基线；调参多、解释性下降 |
| **神经网络** | MLP、深度网络 | 处理非结构化与高交互；需要数据与算力、调参难 |

选择顺序建议：先线性基线 → 再树与集成 → 最后神经网络；每一步都要与前一步比样本外表现，而不是比" 谁更先进"。

来源：家族划分与特点据统计学习教材与 scikit-learn 用户指南整理。

# 【样本 + 可视化】算法性能的定性对照 Algorithm Performance

**Exhibit 5 Learning Curves: Accuracy in Validation and Training Samples vs. Training Sample Size**




这张图给的是" 相对位置"而不是绝对分数：线性模型在小样本、强解释性的场景最稳，树与集成在中等样本、非线性时更强，深度模型要大样本 + 非结构化数据才划算。别把任何一张" 性能榜" 当成选型依据——你的数据不是它的数据。

课堂素材图；解读与帧 46、59 的选型表一致。

# 线性与多元回归 Linear and Multiple Regression

## 作为第一个基线

- 形式简单：$\hat{y} = \beta_0 + \sum_j \beta_j x_j$；
- 系数可读作边际效应（在控制其他变量的条件下）；
- 计算极快，几分钟内就能得到"可达水平"的参照；
- 在样本量远大于变量数、关系近似线性时已经够用。

## 什么时候必须放弃

- 明显非线性或强交互（如面积与地段的交互）；
- 特征数接近或超过样本量（帧 22）；
- 存在大量离群点且未处理（改用稳健回归）。

本讲房价样本的线性基线已经能解释相当一部分变异，但把面积的三次项与面积 $\times$ 地段交互加进去后，样本外 RMSE 明显下降——这就是"基线 $\rightarrow$ 增强"的标准流程。

来源：回归基线的实践据统计学习教材；本讲数值来自程序 l5_03 与 l5_05。

# 惩罚回归：Ridge、Lasso 与 Elastic Net Regularised Regression

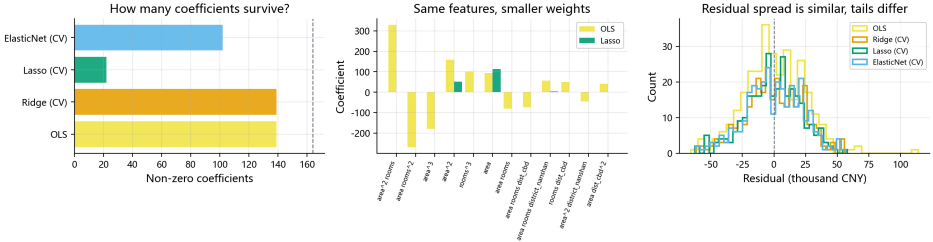
| 方法 | 惩罚项 | 结果与使用场合 |
|---|---|---|
| **OLS** | 无 | 无偏但方差大；$P$ 接近 $N$ 时不可靠 |
| **Ridge** | $\lambda \sum \beta_j^2$ | 系数收缩但不为零；共线性强、特征都有用时首选 |
| **Lasso** | $\lambda \sum \lvert\beta_j\rvert$ | 产生稀疏解（部分系数恰好为零）；需要变量选择时用 |
| **Elastic Net** | 两者混合 | 兼顾稀疏与稳定；特征成组相关时优于纯 Lasso |

三个前提：(1) 惩罚前必须标准化（否则惩罚强度依赖量纲）；(2) $\lambda$ 用交叉验证选（`RidgeCV`/`LassoCV`）；(3) 惩罚会引入偏差，换来更低的方差——这正是帧 35 的偏差-方差权衡。

Penalties trade bias for variance.

来源：三种惩罚的形式与适用场合据统计学习教材；前提与选择方式为本讲补充。

# 【样本 + 可视化】回归族对照 Comparing the Regression Family




同一份 164 维展开特征：Lasso 把绝大多数系数压到零（稀疏），Ridge 保留全部但整体变小；右图显示三种惩罚的残差分布几乎一致——说明在这个样本上，" 稀疏性" 的价值在于可解释与部署成本，而不是精度。

程序 `l5_05_regularized_regression.py` · 系数表 `l5_coefficients.csv` · 结果 `l5_regression_scores.csv`

# 【代码】正则化回归与系数路径 Regularised Regression Paths

```python
poly = PolynomialFeatures(degree=3, include_bias=False) # cubic expansion creates many columns
Xtr_poly = poly.fit_transform(X_train)                  # expand the training features
scaler = StandardScaler().fit(Xtr_poly)                 # scaling is fitted on the training split only
Xtr = scaler.transform(Xtr_poly)                        # scaled training matrix
alphas = np.logspace(-3, 2, 60)                         # grid of penalty strengths, log spaced
models = {                                              # one entry per family
    "OLS": LinearRegression(),                          # no penalty at all
    "Ridge (CV)": RidgeCV(alphas=alphas, cv=5),         # L2 penalty, strength chosen by CV
    "Lasso (CV)": LassoCV(alphas=alphas, cv=5, max_iter=20000, random_state=42), # sparse solutions
    "ElasticNet (CV)": ElasticNetCV(alphas=alphas, l1_ratio=[0.2, 0.5, 0.8], cv=5)} # mixed penalty
for a in alphas:                                        # walk the penalty grid from weak to strong
    r = Ridge(alpha=a).fit(Xtr, y_train)                # ridge at this strength
    l = Lasso(alpha=a, max_iter=20000).fit(Xtr, y_train) # lasso at this strength
    paths_ridge.append(r.coef_)                         # store the ridge coefficients
    paths_lasso.append(l.coef_)                         # store the lasso coefficients
```

要点：缩放 → 展开 → 逐 $\lambda$ 拟合 → 画路径；路径图能直观看出" 哪些变量先被淘汰"。完整程序 `l5_05_regularized_regression.py`。

# K 近邻 K-Nearest Neighbours

## 做法

对新样本，找训练集中距离最近的 $k$ 个邻居，按多数类（分类）或均值（回归）给出预测。属于惰性学习：训练只是存数据，预测时才计算。

## 四个关键点

- 距离度量（欧氏、曼哈顿）与标准化必须一起定；
- $k$ 小 $\to$ 边界复杂、方差大；$k$ 大 $\to$ 边界平滑、偏差大；
- 维度灾难：维度高时" 最近邻" 也不再近；
- 类别不平衡时用距离加权或分层抽样。

本讲双月牙样本的实测（测试集 180 行）：$k = 1$ 准确率 0.839、$k = 15$ 0.911、$k = 99$ 0.861——中间最好，两头都差。

来源：KNN 的性质据统计学习教材；数值来自本讲程序 l5_06 的实测输出。

# 支持向量机 Support Vector Machines

## 核心思想

找一个能把两类分开、并且间隔最大的超平面；允许少量违例（软间隔，由 $C$ 控制）；用核函数把非线性问题映射到高维空间后线性可分。

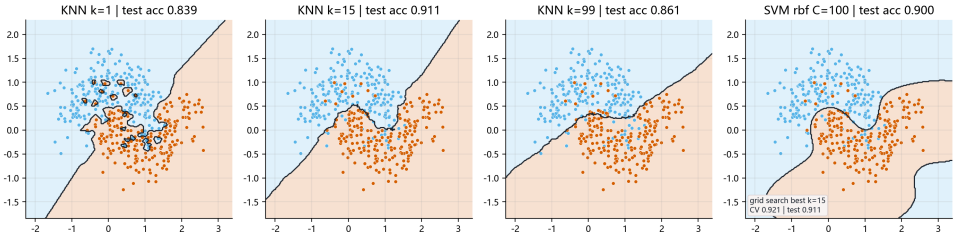
## 调参与注意事项

- $C$ 大：尽量不违例（易过拟合）；$C$ 小：容忍更多违例（更平滑）；
- RBF 核还有 $\gamma$：控制"单个样本的影响半径"；
- 概率输出需要Platt 校准（`probability=True` 会变慢）；
- 标准化几乎总是必须的；样本量很大时训练代价高。

本讲双月牙样本上，RBF 核明显优于线性核；交叉验证给出的最优配置与测试集表现一致（帧 54 的搜索表）。

来源：SVM 与核方法据统计学习教材与 scikit-learn 文档；数值来自程序 l5_06。

# 【样本 + 可视化】KNN 与 SVM 的决策边界 Decision Boundaries




同一份双月牙数据（600 行）：$k = 1$ 边界破碎（测试 0.839）、$k = 15$ 平滑且准确（0.911）、$k = 99$ 过于平滑（0.861）；网格搜索选出 $k = 15$（交叉验证 0.921，测试 0.911）。

程序 `15_06_knn_svm_tuning.py` ·样本 `15_two_moons.csv` ·搜索表 `15_tuning_results.csv`、`15_svm_tuning.csv`

# 决策树与集成学习 Trees and Ensembles

## 决策树（CART）

- 递归地选择"最能降低不纯度的划分"（基尼或熵）；
- 深度与叶节点最小样本数是主要复杂度旋钮；
- 优点：可解释、能处理混合类型与缺失；
- 缺点：方差大——数据稍有变化，树结构就变（帧 57 左图）。

## 集成学习

- Bagging：并行训练多棵深树再平均 → 主要降方差（随机森林）；
- Boosting：串行训练浅树、每棵修正前面的错误 → 主要降偏差（GBDT）；
- 随机森林再加特征子采样，进一步降低树之间的相关性。

本讲房价与信用样本上：单棵树 AUC 0.790 明显落后，随机森林 0.822、梯度提升系 0.809–0.831——集成的收益主要来自降方差。

来源：树与集成的原理据统计学习教材；数值来自本讲程序 l5_07 的实测输出。

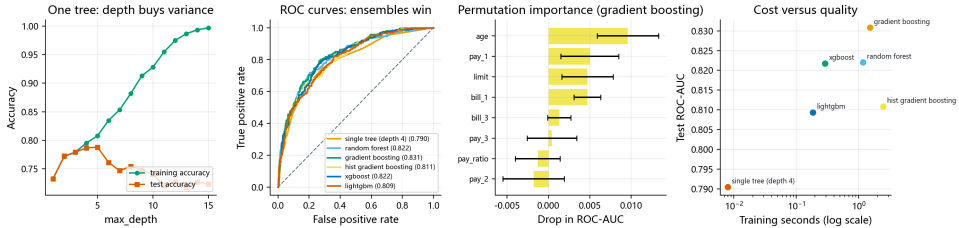
# 梯度提升族：XGBoost、LightGBM、CatBoost Gradient Boosting

| 实现 | 特点 | 本讲实测（信用卡样本，测试集 900 行） |
|---|---|---|
| **经典 GBDT** | 逐棵拟合残差，慢但稳定 | AUC 0.831，训练 1.53 s |
| **scikit-learn HistGB** | 直方图分箱，免独热 | AUC 0.811，训练 2.49 s |
| **XGBoost** | 正则化 + 行列采样，工业标配 | AUC 0.822，训练 0.29 s |
| **LightGBM** | 叶子优先生长，大数据最快 | AUC 0.809，训练 0.19 s |
| **CatBoost** | 类别特征内建处理 | （本讲未含类别特征，未启用） |

三个共同的关键超参数：学习率、树的棵数、树深/叶子数；调参顺序建议：先固定学习率 0.05–0.1，用早停确定棵数，再调结构。

来源：实现特点据各库官方文档；实测数字来自本讲程序 l5_07（同一台机器、同一划分）。

# 【样本 + 可视化】树与集成：代价与收益 Trees versus Ensembles




左一：单棵树加深会训练准确率飙升但测试准确率回落；左二：六种模型的 ROC 曲线；右二：置换重要性显示还款状态三列最重要；右一：训练成本 vs 质量——本样本上 LightGBM 用 0.19 s 拿到 0.809，性价比最高。

程序 `15_07_trees_ensembles.py` ·结果 `15_ensemble_scores.csv` ·重要性 `15_feature_importance.csv`

# 【代码】树与集成：拟合、计时与重要性 Ensembles with Timing

```python
def fit_model(name, model):                          # helper that fits one model and scores it
    pipe = Pipeline([("prep", trees_prep), ("model", model)]) # wrap the estimator with preprocessing
    t0 = time.time()                                 # start the clock
    pipe.fit(X_train, y_train)                       # fit on the training split
    seconds = time.time() - t0                       # record the cost
    proba = pipe.predict_proba(X_test)[:, 1]         # predicted probabilities on the test split
    fpr, tpr, _ = roc_curve(y_test, proba)           # ROC points for the panel
    row = {"model": name,                            # model identifier
          "test_auc": round(float(auc(fpr, tpr)), 4), # area under the ROC curve
          "train_seconds": round(seconds, 3)}        # training cost
    return row, pipe, (fpr, tpr)                     # return everything the figure needs
row, pipe, curve = fit_model("lightgbm", LGBMClassifier(n_estimators=400, # histogram based boosting
    learning_rate=0.06, num_leaves=31, random_state=42, verbose=-1)) # leaf-wise growth, quiet console
perm = permutation_importance(fitted[best_name], X_test, y_test, # model-agnostic importance
                        n_repeats=8, random_state=42, scoring="roc_auc") # eight repeats for a spread
```

要点：计时与打分放在同一个函数里，才能画出"代价—质量"面板；重要性用置换法（模型无关），比内建重要性更可比。

完整程序 `15_07_trees_ensembles.py`。

# 朴素贝叶斯、LDA 与相似度学习 Naive Bayes, LDA, Similarity

## 朴素贝叶斯

假设特征在给定类别下条件独立，用贝叶斯公式分类。训练极快、小样本可用，是文本分类的经典基线；缺点是独立性假设常常不成立（特征高度相关时概率被推得过于极端）。

## 线性判别分析（LDA）

假设各类服从同协方差的高斯分布，寻找使类间/类内方差比最大的方向；自带降维能力，样本量相对特征数偏小时需要正则化版本。

## 相似度学习与其他

- 度量学习：学习一个距离，使同类更近、异类更远（人脸、检索）；
- 最近邻检索：把样本表示为向量，用近似最近邻索引找相似项（衔接 L11 的嵌入与 RAG）；
- 选择原则：先跑一遍简单模型，看它离要求有多远，再决定是否升级。

来源：朴素贝叶斯与 LDA 的假设据统计学习教材；相似度学习与向量检索据信息检索实践。

# 选型决策表 Choosing an Estimator

| 问题 | 数据情况 | 建议起点 |
|---|---|---|
| **数值预测** | 特征相关性强 | PCA 降维后再回归 |
| **数值预测** | 线性、$N \gg P$ | 线性回归 → Ridge/Lasso |
| **数值预测** | 非线性、有交互 | 树与集成（GBDT）→ 神经网络 |
| **分类** | 有标签、线性可分 | 逻辑回归；KNN 或线性 SVM |
| **分类** | 有标签、非线性 | 树与集成；RBF 核 SVM；神经网络 |
| **分类** | 无标签（分组） | 已知组数：k-means；未知组数：层次聚类/DBSCAN |

使用方式：从问题与数据两列出发查找，再回到第 47 页的算法族地图确认代价；任何选择都要用同一套评估协议验证（帧 26–30）。

来源：决策表据估计器选择流程与统计学习教材整理，覆盖原流程中的 PCA、惩罚回归、KNN、SVM、树、集成与聚类分支。

# scikit-learn 估计器地图 Using the Estimator Map

## 怎么用这张图

官方用户指南提供一张交互式估计器地图：从"我有多少样本、要不要预测类别"出发，沿着问题分支一直走到具体估计器，点击即可跳到文档。

## 本课程的用法

- 用地图找候选，用第 60 页的决策表做取舍；
- 候选至少包含一个基线（多数类/均值）与一个简单模型；
- 最终选择由交叉验证的均值与标准差决定，而不是由名气决定。

## 与课程其他讲次的衔接

- L6：线性回归与逻辑回归的闭式解与梯度下降；
- L7：SVM 与核、CART、随机森林、梯度提升、不平衡处理与调参；
- L8：PCA、因子分析、聚类与异常检测；
- L9：深度学习与训练技巧。

来源：估计器地图，scikit-learn 官方用户指南（scikit-learn.org/stable/tutorial/machine_learning_map）。

# 无监督学习的用途 What Unsupervised Learning Is For

## 四类用途

1. 探索：在没有任何假设的情况下看数据结构；
2. 压缩：用更少的维度概括变异（降维）；
3. 分组：把相似对象放到一起（聚类）；
4. 发现异常：找出与大多数不同的个体。

## 最大的困难：没有标准答案

- 没有标签 → 不能用" 准确率" 评价；
- 只能用内部指标（轮廓系数等）与稳定性（重采样后分组是否一致）；
- 最终裁判是业务可解释性与后续用途；
- 因此无监督结果必须与外部变量交叉验证（例如分组是否对应不同的留存率）。

来源：四类用途与评估困难据无监督学习综述与教材整理。

# 主成分分析 Principal Component Analysis

## 在做什么

找到一组互相正交的方向，使数据投影到这些方向后方差最大；第一个方向（PC1）解释最多变异，第二个在与第一个正交的约束下解释次多，依此类推。数学上是对相关系数矩阵做特征值分解。

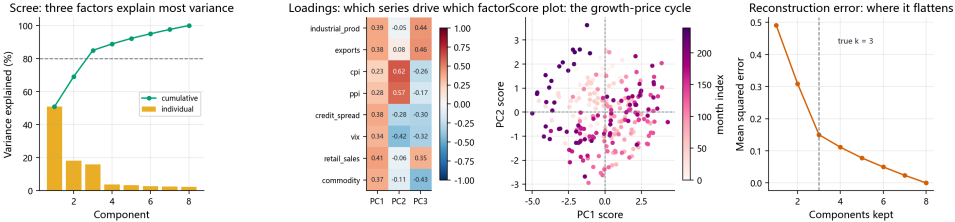
## 使用要点

- 必须先标准化，否则量纲大的变量独占总方差；
- 用碎石图与累计解释率决定保留几个成分；
- 载荷（loadings）说明" 每个成分由哪些变量驱动"；
- 成分的符号不唯一（整体取反等价），解释时要注意。

本讲宏观面板（240 个月 $\times$ 8 个序列，由 3 个潜在因子生成）：前三个成分解释 50.9% + 18.2% + 15.9%，五个成分累计 92.2%。

来源：PCA 的性质据统计学习教材；数值来自本讲程序 l5_08 的实测输出。

# 【样本 + 可视化】PCA 与潜在因子 Principal Components




240 个月、8 个宏观序列（真实结构是 3 个潜在因子）：碎石图在第 3–4 个成分处出现拐点；载荷热力图显示 PC1 由生产与出口驱动、PC2 由物价驱动、PC3 由风险驱动；重构误差曲线也提示 $k \approx 3$。

程序 `15_08_pca_factors.py` ·样本 `15_macro_panel.csv` ·载荷 `15_pca_loadings.csv` ·重构 `15_pca_reconstruction.csv`

# 【代码】PCA 与重构误差 PCA and Reconstruction Error

```
X = panel[cols].values                                  # numeric matrix of observables
X_std = StandardScaler().fit_transform(X)               # standardising is mandatory before PCA
pca = PCA().fit(X_std)                                  # keep all components for the scree plot
evr = pca.explained_variance_ratio_ * 100               # share of variance explained per component
cum = np.cumsum(evr)                                    # cumulative share of variance
scores = pca.transform(X_std)[:, :3]                    # keep the first three component scores
loadings = pd.DataFrame(pca.components_[:3].T, index=cols, # rows are series, columns are components
                        columns=["PC1", "PC2", "PC3"]).round(3) # round for readability
for k in range(1, len(cols) + 1):                       # from one component to all of them
    pca_k = PCA(n_components=k).fit(X_std)              # fit a truncated decomposition
    approx = pca_k.inverse_transform(pca_k.transform(X_std)) # project down and back up
    recon_error.append(float(np.mean((X_std - approx) ** 2))) # mean squared reconstruction error
```

要点：碎石图定个数、载荷定含义、重构误差定损失；三个视角一致时，成分个数才可信。完整程序 `l5_08_pca_factors.py`。

# 因子分析与金融应用 Factor Analysis in Finance

## 与 PCA 的差别

PCA 找的是方差最大的方向（纯数据驱动）；因子分析假设观测由少数潜在因子 + 个体噪声生成，估计的是"共同变动的来源"，因此对变量的共同部分更敏感。

## 金融中的经典用途

- 收益的共同因子：市场、规模、价值（Fama–French 三因子的直觉）；
- 用少量因子解释大量资产的共同变动；
- 因子暴露用于风险归因与组合约束；
- 注意：因子是统计构造，命名与解释需要经济学理由。

方法上的教训：降维后的成分/因子本身没有含义，必须结合载荷与业务知识给它命名，否则"PC1"什么也说明不了。

来源：因子分析与 Fama–French 因子模型据资产定价文献；与 L13 金融专题衔接。

# 非线性降维与嵌入 Non-linear Embeddings

## t-SNE 与 UMAP

- t-SNE：保持局部邻域，簇分得开；簇间距与簇大小不可读；
- UMAP：兼顾局部与全局结构、更快、可对新样本 transform；
- 两者都对参数与随机种子敏感，必须写进图注；
- 用邻域保真度（trustworthiness）而不是" 看起来分得开" 评价。

## 何时用不监督降维

- 可视化：把高维样本画成二维查看结构 (L4 Part 7 已有完整示例)；
- 特征压缩：为下游模型提供更少、更稳的输入；
- 检索：把样本变成嵌入向量，用近似最近邻找相似项（衔接 L11)；
- 陷阱：把降维结果当" 证据"——结论要回到原维度验证。

来源：方法与陷阱据流形学习研究与 L4 第 7 部分的实测经验整理。

# 聚类总览 Four Families of Clustering

| 思路 | 代表方法 | 假设与适用 |
|---|---|---|
| **基于中心** | k-means | 球形、大小相近的簇；需要预设 $k$；快 |
| **基于层次** | 层次聚类（Ward 等） | 不需要预设 $k$；给树状图；$O(n^2)$ 内存 |
| **基于密度** | DBSCAN、HDBSCAN | 任意形状、能识别噪声；对 eps 极敏感 |
| **基于模型** | 高斯混合 + EM | 软分配与概率密度；可给出后验概率 |

本讲行为客户样本（830 行含 30 个离群点）：k-means 与 Ward 都恢复出 4 个生成簇（ARI 1.00 与 0.99），而DBSCAN 在单一 eps 下只分出 2 组、并把 25 个点判为噪声——" 密度聚类能不能用" 取决于簇的密度形状，不是方法高低。

来源：四类聚类思路据聚类分析综述；数值来自本讲程序 l5_09。

# k-means：最常用也最容易被误用 k-means

## 算法与目标

最小化簇内平方和 (inertia)：$\sum_c \sum_{x \in c} \|x - \mu_c\|^2$。交替执行" 分配样本 $\rightarrow$ 更新中心"，直到收敛；用 k-means++ 初始化以降低陷入局部最优的概率。

## 四个常见误用

1. 不做标准化：量纲大的特征独占距离；
2. 用 inertia 选 $k$：inertia 随 $k$ 单调下降，必须看拐点或轮廓系数；
3. 强行解释球形簇：对长条形或密度不均的簇会切错；
4. 忽略初始化：不同随机种子给出不同结果，应设 `n_init` 并报告稳定性。

本讲样本的实测提醒：轮廓系数偏好 $k = 2$（0.568），但真实结构与业务判断都是 4（ARI 1.00）——内部指标只是参考，必须结合外部信息。

来源：k-means 的目标函数与初始化据统计学习教材；数值来自本讲程序 l5_09。

# 层次聚类、DBSCAN 与高斯混合 Hierarchy, Density, Mixtures

## 层次聚类

自底向上合并最近的簇，得到树状图；在合适的距离高度切一刀即得分组。距离定义（单链、全链、平均、Ward）决定结果，Ward 偏好紧凑的球形簇。

## DBSCAN

以邻域半径 eps 与最小点数定义"稠密"；能发现任意形状的簇并识别噪声点，但单一全局 eps 无法同时适配不同密度的簇。

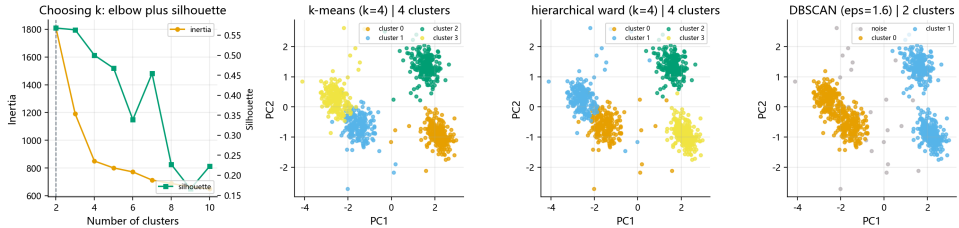
## 高斯混合 + EM

假设数据由若干个高斯分布混合生成：

- E 步：计算每个样本属于各成分的后验概率（软分配）；
- M 步：用后验加权重估均值与协方差；
- 与 k-means 的关系：k-means 是"硬分配 + 等方差球"的特例。

来源：三类方法的表述据聚类分析教材；与 L8 的聚类专题衔接。

# 【样本 + 可视化】三种聚类方法对照 Three Clustering Families




830 个客户（含 30 个离群点）：k-means 与 Ward 恢复出 4 组（ARI 1.00 与 0.99）；DBSCAN 在 eps=1.6 下只找到 2 组、25 个噪声点（ARI 0.50）。左图提醒：轮廓系数偏好 2 组，与生成结构不一致。

程序 `l5_09_clustering.py` ·样本 `l5_customers_behavior.csv` ·对照表 `l5_cluster_scores.csv`

# 【代码】聚类的 k 选择与三种方法 Choosing k and Comparing

```python
for k in k_grid:                                    # loop over the candidates
    km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(X_std) # fit k-means for this k
    sil = silhouette_score(X_std, km.labels_, sample_size=600, random_state=42) # sampled silhouette
    ch = calinski_harabasz_score(X_std, km.labels_)   # variance ratio criterion
    rows.append({"k": k, "inertia": round(float(km.inertia_), 1), # within cluster sum of squares
                 "silhouette": round(float(sil), 4)})   # separation measure
kmeans = KMeans(n_clusters=4, n_init=10, random_state=42).fit(X_std) # centroid based clustering
agglo = AgglomerativeClustering(n_clusters=4, linkage="ward").fit(X_std) # hierarchical, ward linkage
dbscan = DBSCAN(eps=1.6, min_samples=12).fit(X_std)   # density based clustering
mask = lab >= 0                                     # DBSCAN noise points have label minus one
enough = mask.sum() > 10 and len(set(lab[mask])) > 1  # silhouette needs two non-empty clusters
sil = silhouette_score(X_std[mask], lab[mask], sample_size=600, random_state=42) if enough else np.nan # guarded score
ari = adjusted_rand_score(labels_true[labels_true >= 0], lab[labels_true >= 0]) # agreement with truth
```

要点：同时看三个内部指标 + 与已知结构的一致性（ARI）；DBSCAN 的噪声标签（$-1$）在计算指标前必须处理。完整程序 `15_09_clustering.py`。

# 评估、异常检测与关联规则 Evaluation, Anomalies, Rules

## 聚类评估

- 内部指标：轮廓系数、Calinski–Harabasz、Davies–Bouldin；
- 外部指标（有标签时）：ARI、NMI、纯度；
- 稳定性：重采样后分组是否一致（本讲用 ARI 与生成结构对照）。

## 异常检测

孤立森林、LOF、单类 SVM：在没有异常标签时仍可用于筛查，但阈值需要业务确认。

## 关联规则

从交易数据中发现" 一起出现" 的组合 (Apriori、FP-Growth)；关注支持度、置信度、提升度三个量，提升度接近 1 说明只是" 因为都常见"，不是真的关联。

来源：评估指标与三类方法据无监督学习教材与数据挖掘教材；与 L8 的无监督专题衔接。

# 为什么需要神经网络 Why Neural Networks

## 神经网络擅长什么

神经网络（ANN）是一类高度灵活的 ML 算法，在非线性与特征交互突出的任务上表现很好；既用于分类与回归，也是深度学习与强化学习的基础（后两者可以是监督的，也可以是无监督的）。

## 但在表格数据上要先克制

- 表格数据的强基线仍然是树与集成（帧 56–57）；
- 神经网络需要更多数据、更多调参与更多算力；
- 只有在非结构化数据（图像、文本、序列）或强交互场景，神经网络的收益才明显；
- 结论：先跑集成，再决定要不要上网络。

来源：神经网络定位据金融机器学习教材；表格数据与深度模型的取舍据近年实证对比研究。

# 从多元回归到神经网络 From Regression to Networks

## 多元回归（对照）

用 4 个输入特征预测 1 个输出：$\hat{y} = \beta_0 + \beta_1 x_1 + \cdots + \beta_4 x_4$。预测值是输入的线性组合。

## 单隐层神经网络

- 由节点（圆圈）与链接（箭头）构成；
- 三类层：输入层（每个特征一个节点）、隐层（学习发生的地方）、输出层（把结果传出网络）；
- 关键差别：隐层节点把输入非线性地变换成新值，再组合成输出——没有非线性，多层就退化成一层。

本讲的实测对照（双月牙样本）：手写两层网络测试准确率 0.929，scikit-learn 的同宽度网络 0.933——机制一致时，实现细节决定最后几个千分点。

来源：结构与表述据金融机器学习教材的神经网络章节；数值来自本讲程序 l5_10。

# 一个节点的内部：求和与激活 Inside One Neuron

## 两个功能部件

1. 求和算子：把收到的输入各乘一个权重再相加，得到总净输入；
2. 激活函数：把总净输入变换成该节点的最终输出。

打个比方，激活函数像调光开关，把输入的强度调高或调低。

## 激活函数必须是非线性的

| 函数 | 特点 |
|---|---|
| sigmoid | 输出 0–1，两端梯度小（易梯度消失） |
| tanh | 输出 $-1$–1，零中心 |
| ReLU | $\max(0, z)$，计算快、缓解梯度消失；负半轴梯度为零 |
| softmax | 输出层用于多分类，输出和为 1 |

非线性意味着输出的变化率随输入水平而不同——这正是神经网络能拟合弯曲关系、而线性模型不能的原因。

来源：节点机制与激活函数的表述据金融机器学习教材；函数对照为通用整理。

# 结构就是超参数 Architecture as Hyper-Parameters

## 从 4-5-1 说起

一个输入层 4 个节点、1 个隐层 5 个节点、输出层 1 个节点的网络，"4、5、1" 这三个数字就是超参数——它们决定网络的结构。更复杂的网络可以有多个隐层、每层不同宽度。

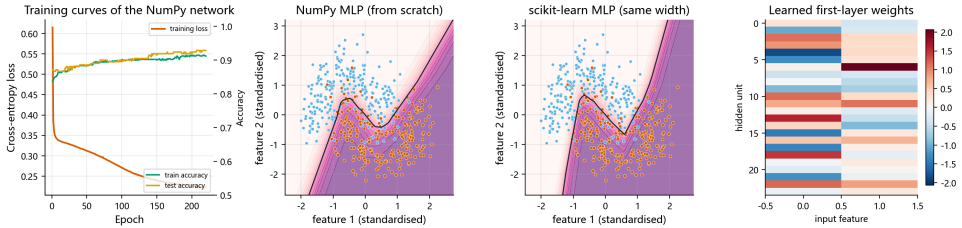
## 主要超参数清单

| 超参数 | 作用 |
|---|---|
| 层数与宽度 | 模型容量 |
| 学习率 | 每步更新幅度 |
| 批大小 | 梯度估计的噪声与速度 |
| 正则化 | dropout、权重衰减、早停 |
| 轮数 epoch | 训练多久（配合早停） |

纪律：网络结构也要进交叉验证。本讲的手写网络固定一个隐层 24 个单元，就是为了让" 机制" 而不是" 调参" 成为重点。

来源：超参数清单据深度学习实践与金融机器学习教材整理。

# 【样本 + 可视化】两层网络的训练过程 Training Two-Layer Net




左一：手写网络的损失下降、训练与测试准确率上升（双月牙样本）；中两格：手写网络与 scikit-learn 同宽度网络的决策边界几乎一致（测试准确率 0.929 vs 0.933）；右一：第一层权重热力图——网络学到了”特征组合”。

程序 `15_10_mlp_numpy.py` ·训练历史 `15_mlp_history.csv` ·样本 `15_two_moons.csv`

# 前向传播 Forward Propagation

## 从输入到输出

输入层的值经链接传给隐层节点：每条链接带一个权重，节点把加权值求和，再经激活函数"决定是否发放"。负的净输入经 S 形函数后接近 0（不发放），正的净输入接近 1（发放）；激活后的值继续向前传给下一层，直到输出层。

## 矩阵写法

$$z_1 = XW_1 + b_1, \quad a_1 = \mathrm{ReLU}(z_1)$$

$$\hat{y} = \mathrm{softmax}(a_1 W_2 + b_2)$$

这就是本讲手写实现中的两行代码；一层的输出是下一层的输入，如此而已。

来源：前向传播的表述据金融机器学习教材的神经网络章节；矩阵形式为本讲补充。

# 【代码】NumPy 前向与反向传播 Forward and Backward

```python
def forward(Xb, params):                          # forward propagation through the network
    W1, b1, W2, b2 = params                       # unpack the parameters
    z1 = Xb @ W1 + b1                             # linear combination of the inputs
    a1 = np.maximum(z1, 0.0)                      # ReLU activation of the hidden layer
    logits = a1 @ W2 + b2                         # linear output layer
    exp = np.exp(logits - logits.max(axis=1, keepdims=True)) # stabilised exponentials for softmax
    proba = exp / exp.sum(axis=1, keepdims=True)  # class probabilities
    return logits, a1, proba                      # return intermediate values for back propagation
def loss_and_grads(Xb, yb, params, l2=1e-4):      # cross-entropy loss plus its gradients
    logits, a1, proba = forward(Xb, params)       # run the forward pass
    loss = -np.log(np.clip(proba[np.arange(n), yb], 1e-12, 1.0)).mean() # mean cross-entropy
    dlogits = proba.copy()                        # gradient of the loss with respect to the logits
    dlogits[np.arange(n), yb] -= 1.0              # subtract one at the true class
    dW2 = a1.T @ dlogits + 2 * l2 * W2            # gradient for the second weight matrix
    dz1 = da1 * (a1 > 0)                          # ReLU derivative: one where the unit fired
    dW1 = Xb.T @ dz1 + 2 * l2 * W1                # gradient for the first weight matrix
```

要点：前向算出中间值，反向按链式法则逐层回传；交叉熵对 logits 的梯度恰好是" 预测减真实"，这让实现异常简洁。完整程序 `15_10_mlp_numpy.py`。

# 训练循环与优化器 Training Loop and Optimisers

## 一次迭代发生了什么

1. 取一个小批量（mini-batch）样本；
2. 前向计算预测与损失；
3. 反向计算各参数的梯度；
4. 沿梯度更新权重（学习率决定步长）；
5. 重复直到遍历完数据（一个 epoch），多轮直到收敛。

## 优化器家族

- SGD：最简单的梯度下降，需仔细调学习率；
- Momentum：积累方向，减少震荡；
- Adam：自适应步长，通常收敛更快（本讲 sklearn 对照用 Adam）；
- 早停：验证误差不再下降就停，是最有效的正则化之一。

调试顺序：先让模型在小批量上过拟合（确认能学），再正规化；损失不下降先查学习率与数据缩放，而不是先换结构。

来源：训练循环与优化器据深度学习实践教材；调试顺序为本讲补充。

# 深度学习网 Deep Learning Nets

## 定义与规模

带有很多隐层的神经网络被称为深度学习网 (DLN) ——至少 3 层，常见超过 20 层，是这一轮人工智能革命的骨干。DLN 推动了图像、模式与语音识别等复杂任务的进展。

## 它是怎么工作的

输入特征逐层经过带权重的非线性函数，每层输出通常落在 $(0,1)$ 或 $(-1,1)$ 区间；最后一层给出" 样本属于各个类别的概率"，取概率最高的类别作为预测。训练时用大数据集确定权重，以最小化指定损失函数。

2026 视角：规模与成本的取舍变了——小模型微调或提示 + 检索常常比从头训练更划算（帧 86–87、95）。

来源：深度学习网的表述据金融机器学习教材；2026 取舍为课程补充。

# 强化学习 Reinforcement Learning

## 核心设定

智能体（agent）应当采取一系列动作，以最大化长期回报，同时受环境约束。以 AlphaGo 为例：虚拟棋手（智能体）用控制台指令（动作），依据屏幕信息（环境），最大化自己的得分（回报）。

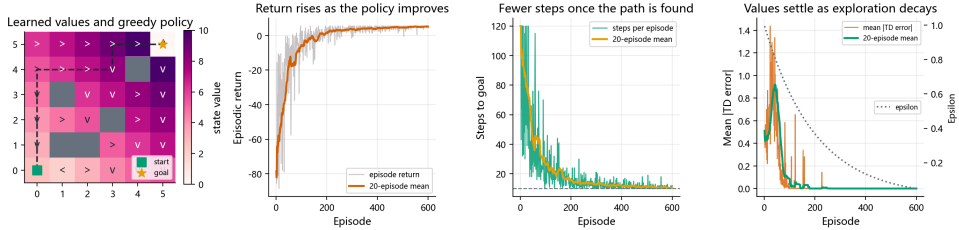
## 与监督学习的三点不同

1. 没有每个观测的直接标签；
2. 没有即时反馈——好动作的收益可能在很多步之后；
3. 算法必须观察环境、试验新动作、复用历史经验，其中一些动作在短期内看起来并不优。

关键权衡是探索与利用：本讲的 $\varepsilon$-贪心策略一开始几乎随机探索，随后逐渐转向利用学到的价值（帧 84 的曲线）。

来源：强化学习设定与 AlphaGo 案例据金融机器学习教材；探索-利用的表述为通用整理。

# 【样本 + 可视化】Q-learning 在网格世界里学会走路




6×6 网格、五个障碍：左上角是学到的状态价值热力图与贪心策略箭头（虚线为最优路径，长度 11 步）；右三格显示回合回报上升、步数下降、TD 误差随探索率衰减而收敛（最后 20 回合平均回报 5.20，平均 10 步到达目标）。

程序 `l5_11_rl_gridworld.py` ·训练历史 `l5_rl_returns.csv` ·无标签、无即时反馈

# 【代码】Q-learning：一个能跑通的强化学习例子

```python
def step(state, action):                                # one environment transition
    r, c = state[0] + action[0], state[1] + action[1] # candidate next cell
    if not (0 <= r < SIZE and 0 <= c < SIZE):          # moving off the grid is not allowed
        return state, -1.0                              # stay in place and pay the step cost
    if (r, c) in WALLS:                                 # moving into a wall is not allowed
        return state, -1.0                              # stay in place and pay the step cost
    if (r, c) == GOAL:                                  # reaching the goal ends the episode
        return (r, c), 10.0                             # terminal reward
    return (r, c), -0.5                                 # small negative reward encourages speed
for episode in range(1, episodes + 1):                  # loop over the episodes
    if rng.random() < eps:                              # explore with probability epsilon
        a = int(rng.integers(len(ACTIONS)))             # pick a random action
    else:                                               # otherwise exploit the current knowledge
        a = int(np.argmax(Q[state[0], state[1]]))       # greedy action from the Q table
    best_next = 0.0 if next_state == GOAL else float(np.max(Q[next_state[0], next_state[1]])) # bootstrap value
    td_target = reward + gamma * best_next              # the Q-learning target
    Q[state[0], state[1], a] += alpha * (td_target - Q[state[0], state[1], a]) # value update
```

要点：奖励设计决定行为（本讲用 $-0.5$ 的步进代价鼓励走短路），$\varepsilon$ 衰减控制探索；这就是 RLHF 之外最小可用的强化学习骨架。完整程序 `l5_11_rl_gridworld.py`。

# 自监督、迁移学习与微调 Self-Supervision and Transfer

## 三个概念

- 自监督：用数据自身构造监督信号（预测被遮住的词、判断两张图是否同源）；
- 迁移学习：先在大规模数据上预训练，再迁移到小数据任务；
- 微调：在目标任务上继续更新（部分或全部）权重。

## 三种用法怎么选

| 方式 | 适用 |
|---|---|
| 特征提取 | 数据很少、算力有限：冻结主体，只训分类头 |
| 部分微调 | 中等数据：只微调最后几层 (PEFT/LoRA) |
| 全量微调 | 数据充足、任务与预训练差异大 |
| 提示 + 检索 | 无法微调或需要最新知识 (衔接 L11) |

决策依据不是" 哪个更先进"，而是数据量、算力预算、更新频率与延迟要求——下一帧的 2026 取舍表给出判据。

来源：自监督与迁移学习的分类据深度学习与基础模型文献；用法选择为本讲整理。

# 2026 取舍：AutoML、基础模型与自建模型 Build or Reuse

| 路线 | 优势 | 代价与风险 |
| --- | --- | --- |
| **自建监督模型** | 完全可控、可解释、推理便宜 | 需要标注数据与工程投入；泛化范围窄 |
| **AutoML（自动搜索）** | 快速得到强基线、省调参人力 | 搜索算力成本；可解释性与可复现性要额外管理 |
| **基础模型微调** | 少样本即可迁移、效果好 | 算力与数据治理成本；版本与漂移管理复杂 |
| **提示 + 检索** | 无需训练、更新最快 | 依赖外部服务；延迟与成本波动；需要评测防幻觉 |

本课程的立场：先用简单模型把评估协议立起来，再按" 数据量 × 更新频率 × 延迟预算" 三问决定是否升级到微调或基础模型（衔接 L10、L11）。

来源：取舍表据 AutoML 与基础模型工程实践整理；与课程 L1、L11 的 AI 版图和 NLP 内容衔接。

# 可解释性的三类方法 Three Families of Explainability

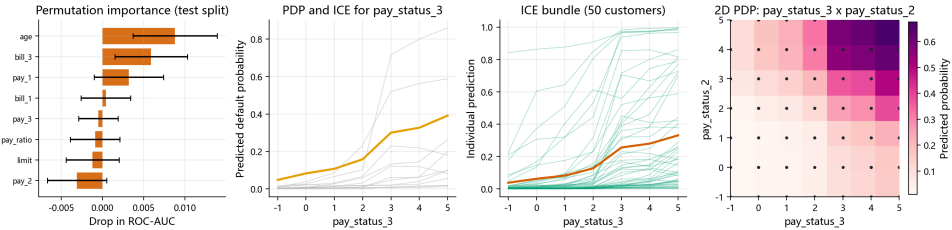
| 类型 | 代表方法 | 说明与注意 |
|---|---|---|
| **内在可解释** | 线性/逻辑回归、浅树、规则集 | 结构与系数本身可读；精度通常低于集成 |
| **模型无关** | 置换重要性、PDP、ICE | 对任意模型可用；置换会破坏特征相关性 |
| **归因方法** | SHAP、LIME、梯度类方法 | 给出单样本贡献；近似方法的稳定性需检验 |

三条纪律：(1) 解释不等于因果；(2) 相关特征会分摊重要性，别把" 重要性低" 当成" 没用"；(3) 解释也要记录与版本化 (帧 93 的模型卡)。

Explain the model, not the world.

来源：三类方法据统计机器学习可解释性文献；纪律为本讲补充。

# 【样本 + 可视化】解释梯度提升模型 Explaining a Boosting Model




信用卡样本（测试 ROC-AUC 0.809、Brier 0.158）：置换重要性显示近三个月的还款状态最重要（0.094 / 0.092 / 0.090）；PDP 给出平均边际效应，ICE 显示个体曲线并不重合；右下角二维 PDP 揭示逾期次数与金额之间的交互。

程序 `l5_12_interpretability_modelcard.py` ·重要性 `l5_model_importance.csv` ·模型卡 `artifacts/model_card_l5_credit.md`

# 【代码】解释性可视化与模型卡 Explanation and Model Card

```
perm = permutation_importance(model, X_test, y_test, n_repeats=10, # shuffle one feature at a time
                              random_state=42, scoring="roc_auc") # measure the drop in ROC-AUC
importance = pd.DataFrame({"feature": X.columns,          # feature names
                           "importance": perm.importances_mean.round(4), # mean drop in ROC-AUC
                           "std": perm.importances_std.round(4)}) # spread across the ten repeats
pd_one = partial_dependence(model, X_test, [top2[0]], grid_resolution=40, kind="average") # one dimensional PDP
for _, row in sample_rows.iterrows():                # loop over the sampled customers
    tmp = pd.DataFrame([row] * len(grid_values), columns=X.columns) # repeat the row along the grid
    tmp[top2[0]] = grid_values                       # vary the feature of interest
    ice_curves.append(model.predict_proba(tmp)[:, 1]) # store the individual curve
card = f"""# Model card -- Lecture 5 credit default demo         # one page markdown model card
## Intended use ... ## Model ... ## Data ... ## Metrics ... ## Limitations
"""                                                  # end of the model card text
dump_artifact(card, "model_card_l5_credit.md")       # write the model card
write_json({"lecture": "L5", "seed": 20260506,       # run log metadata for reproducibility
            "metrics": {"roc_auc": ..., "brier": ...}}, "l5_run_log_12.json") # write the run log
```

要点：重要性 + PDP + ICE + 二维交互四张图一起看，再把用途、数据、指标、限制写进模型卡；运行记录落盘才能复现。完整程序 `l5_12_interpretability_modelcard.py`。

# 数据泄漏清单 The Leakage Checklist

## 五类泄漏与自检问题

1. 目标泄漏：特征是否只在结果已知后才能获得？（本讲把它做出 AUC 1.000）
2. 选择泄漏：特征选择/降维是否在划分之前做了？
3. 预处理泄漏：标准化、插补是否在划分之前拟合？
4. 重复行泄漏：同一实体是否同时出现在训练与测试？
5. 时间泄漏：是否用未来信息预测过去？

## 三条工程措施

- 一切预处理放进流水线（帧 29）；
- 先划分、后处理，且切分逻辑写进脚本；
- 用时间切分与分组切分替代随机切分；
- 交付前做一次" 如果我只知道当时的信息，这个特征还会有吗"的灵魂拷问。

来源：五类泄漏据机器学习工程实践整理；本讲的量化演示见程序 l5_02 与样本页 30 页。

# 公平性与偏差 Fairness and Bias

## 偏差从哪来

- 历史偏差：训练数据反映了过去的歧视性决策；
- 代理变量：邮编、学历等与受保护属性高度相关；
- 测量偏差：标签本身由有偏的流程产生（如只对某群体严格审查）；
- 样本偏差：某些群体在数据中代表性不足。

## 常用公平性指标与权衡

| 指标 | 含义 |
| --- | --- |
| 人口均等 | 各组正类预测率相同 |
| 机会均等 | 各组在真实正类上的召回相同 |
| 预测均等 | 各组精确率相同 |

这些指标通常不能同时满足，必须结合业务与合规目标选择并写进模型卡。

来源：公平性指标据机器学习公平性文献；权衡说明为通用结论（与 L15 营销与国际商务伦理衔接）。

# 模型卡：一页说清你的模型 The Model Card

## 模型卡的六个部分

1. 用途：做什么、不做什么（明确禁止的用法）；
2. 模型：算法、超参数、版本；
3. 数据：来源、规模、正类比例、已知偏差；
4. 指标：样本外指标与不确定性；
5. 限制：失效场景、已知风险；
6. 伦理考量：受保护属性、潜在的差别影响。

## 为什么必须写

- 让接手的人知道这份模型能用在哪；
- 让审计与合规有据可查（帧 98）；
- 迫使自己回答"失败时会怎样"；
- 本讲程序会自动生成一份模型卡草稿（`artifacts/`）。

来源：模型卡的组成据模型文档（Model Cards）研究；本讲给出可直接使用的模板。

# 从脚本到系统：MLOps 基础 From Script to System

## 四件事

1. 版本：数据、代码、模型三者版本必须能对应（本讲用种子 + JSON 运行记录）；
2. 实验跟踪：记录参数、指标、产物（MLflow、Weights & Biases 的职责）；
3. 监控：输入分布漂移、预测分布漂移、业务指标；
4. 回滚：发现退化能一键切回上一个版本。

## 本讲的轻量做法

- 每个程序输出`l5_run_log_*.json`：脚本名、种子、数据摘要、指标、产物清单；
- 模型卡为 Markdown，随代码一起进仓库；
- 生产环境的 MLflow/W&B 只是把同一套信息集中管理，不是另一个世界。

来源：MLOps 四要素据机器学习工程实践；轻量做法为本讲为教学环境设计的替代方案。

# AutoML：省调参，但不省判断 AutoML and Its Costs

## AutoML 能做什么

自动搜索特征处理、模型族与超参数的组合，按给定指标选出最优配置（Optuna 等工具做贝叶斯搜索）。它把" 调参人力" 换成" 搜索算力"。

## 三个必须自己负责的事

1. 评估协议：搜索用的交叉验证必须与最终报告一致，否则搜索本身就是泄漏；
2. 选择偏差：试了很多配置后，最优分数天然乐观，需要在独立测试集上复核；
3. 可复现与可解释：搜索过程与最终配置要记录，否则无法交付（帧 93–94）。

一句话：AutoML 让" 找到好模型" 变便宜，但" 证明它真的好" 仍然要靠人。

来源：AutoML 的能力与风险据自动机器学习与超参数优化文献；纪律为本讲补充。

# 成本与延迟：模型的隐藏约束 Cost and Latency

## 三类成本

- 训练成本：一次要多久、多久重训一次（本讲六种模型 0.008–2.49 秒）；
- 推理成本：单次预测的算力与费用；
- 错误成本：误报与漏判的业务损失（帧 44）。

## 延迟预算怎么定

- 实时风控：几十毫秒级 → 线性模型或小树模型；
- 批量营销：小时级 → 梯度提升或集成；
- 离线研究：天级 → 神经网络与大模型；
- 兜底策略：降级到简单模型，保证可用性。

来源：成本结构据机器学习系统实践；本讲数值来自程序 l5_07 的实测计时（同机同划分）。

# 指标被优化之后 After the Metric Is Optimised

## 指标失真的三种机制

- 过拟合测试集：反复看测试分数做选择；
- 代理指标偏离目标：优化点击率却损害长期价值；
- 分布漂移：训练时的分布与线上不再一致。

## 四条对策

1. 保留一个从未看过的测试集，只在最后用一次；
2. 监控输入与预测分布的漂移（帧 94）；
3. 报告校准与区间，而不只是点估计；
4. 定期用新的时间段数据做外部复核，并记录退化速度。

来源：指标失真与漂移监控据机器学习评测与生产实践文献；对策为本讲整理。

# 合规与高风险场景 Compliance in High-Risk Settings

## 高风险应用的共同要求

- 可追溯：数据、代码、模型版本可对应（帧 94）；
- 可解释：给出影响决策的主要因素（帧 89）；
- 人工复核：对高影响决策保留人工介入；
- 记录与报告：模型卡、评测结果、已知限制；
- 持续监控：上线后评估偏差与漂移。

## 教育场景的红线

- 不用模型自动判定学生的最终成绩或资格；
- 不把敏感个人信息作为特征或标签；
- AI 生成的分析结果必须标注并复核（与课程 AI 使用规范一致）；
- 涉及个人数据的研究，遵守知情同意与最小必要原则。

来源：高风险场景要求据 EU AI Act 等合规框架的通行要点；教育场景红线据本课程与学校规范。

# 交付前自查清单 Pre-Delivery Checklist

1. 有基线吗？（多数类/均值）
2. 划分方式写清了吗（随机/分层/时间/分组）？
3. 预处理是否在流水线内？
4. 有没有泄漏（五类逐条检查）？
5. 指标是否匹配问题与代价？
6. 概率校准了吗？阈值怎么定的？
7. 结果能解释吗（重要性/PDP/ICE）？
8. 模型卡与运行记录齐了吗？
9. 成本与延迟在预算内吗？
10. 失败时会怎样，兜底方案是什么？

用法：把这十条当作提交前最后一分钟的自检；任何一条答不上来，就先补上再交。

来源：清单综合本讲 Part 2–7 的全部内容，可直接用于课程项目与工作实践。

# 工具栈 The Tool Stack

| 环节 | 工具（本课程使用） |
|---|---|
| **数据处理** | pandas、NumPy（L3、L4 已熟悉） |
| **建模与评估** | scikit-learn（流水线、模型、指标、校准、可解释性） |
| **梯度提升** | XGBoost、LightGBM（CatBoost 处理类别特征） |
| **调参与搜索** | `GridSearchCV`、`RandomizedSearchCV`、Optuna |
| **实验跟踪** | 本讲用 JSON 运行记录 + 模型卡 演示；生产可用 MLflow、Weights & Biases |
| **可视化与解释** | matplotlib、plotly（L4）；置换重要性、PDP/ICE（可选 SHAP） |

课程仓库 `website/code/lecture05/` 提供 12 个程序：每一行都有英文注释，一条命令 `python run_all.py` 即可重建全部样本、图表与实验记录。

来源：工具选择对应课程大纲第 5 讲工具条目：scikit-learn、XGBoost/LightGBM、MLflow、Weights & Biases。

# 第 6 周任务：项目选题说明（一页）Project One-Pager

## 模板（四段）

1. 问题：是预测还是因果？预测什么、给谁用、错了代价多大？
2. 数据：来源与许可、规模、标签如何得到、已知偏差；
3. 方法：基线 + 候选模型；划分方式；指标与阈值规则；解释方式；
4. 预期产出：图表清单、模型卡、可复现脚本、结论的适用边界。

## 评分关注点

- 问题是否可检验（指标能算出来）；
- 数据是否合法可得（来源写明）；
- 评估协议是否防泄漏；
- 是否给出基线与失败时的兜底。

写法建议：用第 99 页的十条自查清单反向检查这份说明——每条都能在说明里找到对应句子，就算合格。

来源：任务与提交要求见课程大纲第 6 周安排；模板为本讲提供。

# 常见错误 Top 10 Common Mistakes

1. 不做基线，直接上复杂模型；
2. 用测试集调参或反复看测试分数；
3. 预处理写在划分之前（泄漏）；
4. 随机切分时间序列或分组数据；
5. 只看准确率，忽略不平衡与代价；
6. 报了概率却不校准、不定阈值；
7. 把特征重要性当成因果效应；
8. 用聚类结果直接下业务结论而不做外部验证；
9. 没有模型卡与运行记录，无法复现；
10. 忽略成本与延迟，模型上不了线。

这十条与第 99 页的自查清单一一对应：先看错误清单，再用清单验收。

来源：错误清单综合本讲各部分的实测演示与常见评审意见。

# 术语对照与参考资料 Glossaries and References

| 中文 | English |
|---|---|
| 特征 / 标签 | feature / label |
| 超参数 | hyper-parameter |
| 训练·验证·测试 | train/validation/test |
| 交叉验证 | cross-validation |
| 偏差 / 方差 | bias / variance |
| 过拟合 / 欠拟合 | overfitting / underfitting |
| 正则化 | regularisation |
| 数据泄漏 | data leakage |
| 集成学习 | ensemble learning |
| 梯度提升 | gradient boosting |
| 降维 / 聚类 | dimension reduction / clustering |
| 轮廓系数 | silhouette score |
| 校准 | calibration |
| 强化学习 | reinforcement learning |
| 模型卡 | model card |

## 参考书与资料

- Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow* (3rd ed.), Ch.1–3（大纲指定）
- Mitchell, T. (1997). *Machine Learning*（E/T/P 定义）
- Hastie, Tibshirani & Friedman. *The Elements of Statistical Learning*, Ch.7
- James et al. *An Introduction to Statistical Learning*
- Athey, S. (2017, *Science*)；Mullainathan & Spiess (2017, *JEP*)：预测与因果的分工
- scikit-learn 用户指南与估计器地图；XGBoost / LightGBM 文档；Optuna；MLflow
- 模型卡（Mitchell et al., 2019）；EU AI Act 高风险清单；Kaggle Learn

来源：参考书对应课程大纲第 5 讲阅读要求；原课件链接与工具条目已并入工具栈一页。

# 谢谢！

Thank you · Q & A

本讲小结：四类范式 → 划分与评估协议 → 偏差-方差与指标 → 监督与无监督算法族 → 神经网络与强化学习 → 可信交付

课堂检查点：用三句话讲清你的模型——预测什么、怎么评、错在哪。

样本与程序：`website/code/lecture05`（12 个程序，逐行英文注释） `website/data/lecture05`（样本 + 模型卡 + 运行记录） `website/figures/lecture05`（14 张图）

下一讲预告：Lecture 6 Logistic Regression and Linear Classifier （从线性回归到交叉熵与梯度下降）

任课教师：戴谢尔 (Xieer Dai, David) · Xieer.dai@szu.edu.cn · 四方楼南 209